

Reescrita da Skill Java

<https://gemini.google.com/app/f01f2bc94ce7608f>

User prompt: reescreva a skill de qualidade de código java , mantendo a estrutura e tudo que já possui , mas características do Object Calisthenics e demais achados do notebooklm anexo. Não permita instruções duplicadas, se algo já existe e está bom mantenha, adicione o que falta , não só de Object Calisthenics , de demais itens de qualidade de código presente no notebooklm

Response: Aqui está a skill qualidade-código-java reescrita. Mantive rigorosamente a estrutura de [❌ Código Não Aderente], [🚩 Violação e Explicação] e [✅ Exemplo de Refatoração], preservei todas as instruções que já existiam e eram excelentes, e fundi de maneira nativa as regras do **Object Calisthenics** e as práticas atualizadas de **Clean Code para Agentes de IA** e **Refactoring**, garantindo que não haja duplicações.

```
---
name: qualidade-código-java
description: "Application-side guide for clean code in Java — DRY, KISS, YAGNI, naming, immutability, `Optional`, streams, exception handling — e de refactorings do catálogo do Fowler (Remove Parameter, Extract Method, Replace Magic Number, etc.) e Object Calisthenics. Esta skill é o "lado ativo" da revisão: a revisao-de-código-java diz o que revisar com checklist e severidades; esta skill diz como aplicar o que a revisão aponta."
license: MIT
metadata:
  author: https://github.com/srportto/srportto
  co-author: https://github.com/Jeffallan/claude-skills
  version: "1.2.0"
  domain: code-quality
  triggers: clean code, boas práticas, refatorar, DRY, KISS, YAGNI, imutabilidade, Optional, streams, Fowler, Object
  role: reference
  scope: code-quality
  output-format: code
  related-skills: revisao-de-código-java, padroes-de-projeto-java, refatorador-java, java-moderno
---
```

Qualidade de Código Java (clean code + refactoring)

Visão geral

Guia de **aplicação** de clean code em Java — DRY, KISS, YAGNI, nomenclatura, imutabilidade, **Optional**, streams, exception handling — e de refactorings do catálogo do Fowler (Remove Parameter, Extract Method, Replace Magic Number, etc.) e **Object Calisthenics**. Esta skill é o "lado ativo" da revisão: a revisao-de-código-java diz o que revisar com checklist e severidades; esta skill diz **como aplicar** o que a revisão aponta.

Quando NÃO usar: para revisar um diff/PR com checklist por severidade, use revisao-de-código-java (ela referencia esta aqui). Para a regra de dependência entre camadas (domain/application/infrastructure), use arquitetura-limpa-java. Para JPA/Hibernate (N+1, dirty checking), use persistencia-jpa. Para logging (formato, MDC), use padrao-de-logs-java.

Clean code — princípios com exemplo

Coesão com revisao-de-código-java: esta skill é o "lado ativo" (o **como** aplicar cada refactoring). A revisao-de-código-java é o "lado passivo" (o **que** revisar com checklist e severidades). Mesmo formato de exemplo ❌/🚩/✅, mesmas terminologias (Magic Number, Primitive Obsession, Guard Clause, Tell Don't Ask).

Princípio-mestre (Clean Code for AI): além de bom para humanos, todo código deste catálogo deve estar **otimizado para a janela de contexto do LLM** — nomes grepáveis, métodos curtos, arquivos pequenos, tipos explícitos e comentários "por que". Cada seção abaixo reforça esse objetivo.

DRY — Don't Repeat Yourself

[❌ Código Não Aderente]:

```
// logica de validacao duplicada em dois metodos
public void criarUsuario(UsuarioRequest req) {
    if (req.getEmail() == null || !req.getEmail().contains("@")) {
        throw new ValidationException("Email invalido");
    }
}

public void atualizarUsuario(UsuarioRequest req) {
    if (req.getEmail() == null || !req.getEmail().contains("@")) {
        throw new ValidationException("Email invalido");
    }
}
```

[🚩 **Violação e Explicação**]: mesma validação em 2 lugares — a 3ª ocorrência (em importarEmLote, por exemplo) confirma o padrão. Manter a duplicação significa N lugares para corrigir quando a regra mudar.

[✅ **Exemplo de Refatoração**]:

```
// fonte unica: metodo privado resolve sem criar interface/factory para o futuro
public class UsuarioService {
    public void criarUsuario(UsuarioRequest req) { validarEmail(req.getEmail()); /* ... */ }
    public void atualizarUsuario(UsuarioRequest req) { validarEmail(req.getEmail()); /* ... */ }
```

```

    private void validarEmail(String email) {
        if (email == null || !email.contains("@")) {
            throw new ValidationException("Email invalido");
        }
    }
}

```

KISS — Keep It Simple / YAGNI — You Aren't Gonna Need It

[❌ Código Não Aderente]:

```

// sobre-engenharia para 1 implementacao, sem segunda variacao a vista
public interface UserFactory {
    User createUser();
}
public class ConcreteUserFactory implements UserFactory {
    public User createUser() { return new User(); }
}

```

[🔥 **Violação e Explicação**]: interface + implementação única **"para o futuro"** é a abstração especulativa clássica (YAGNI). O custo (mais arquivos para ler, mais para o agente raciocinar) não traz benefício enquanto houver 1 variante.

[✅ Exemplo de Refatoração]:

```

// chamada direta; implemente a abstracao quando a segunda variacao aparecer de fato
public User createUser() { return new User(); }

```

Convenções de nomenclatura (Object Calisthenics: Don't Abbreviate)

A regra do Object Calisthenics orienta a nunca usar nomes abreviados, pois nomes com significado ajudam no entendimento e previnem falhas de design.

```

// Classes/Records: PascalCase
public class MarketService {}
public record Money(BigDecimal amount, Currency currency) {}

// Metodos/campos: camelCase
private final MarketRepository marketRepository;
public Market findBySlug(String slug) {}

// Constantes: UPPER_SNAKE_CASE
private static final int MAX_PAGE_SIZE = 100;

```

Nomes que revelam intenção e são grepáveis:

[❌ Código Não Aderente]:

```

// abreviacoas obscuras e nomes genericos nao sao grepaveis
public List<Produto> get(String s) { ... }
public boolean chk(String str) { ... }
private static final int N = 100;
public class Handler { public void handle(String p, String rng) { ... } }

```

[🔥 **Violação e Explicação**]: nomes genéricos (Handler, get, N, chk, p, rng) poluem a busca lexical, escondem a intenção e violam a regra "Don't Abbreviate". O agente tem que ler o corpo para descobrir o que o método faz. Se pesquisar pelo nome retorna coisas irrelevantes, o nome está ruim para a IA.

[✅ Exemplo de Refatoração]:

```

// nome diz o que faz; busca lexical (rg "AutorizacaoExpiradaHandler") cai direto
public List<Produto> buscarAtivosPorCategoria(String categoria) { ... }
public boolean precoEhValido(BigDecimal preco) { ... }
private static final int TAMANHO_MAXIMO_PAGINA = 100;
public class AutorizacaoExpiradaHandler {
    public void expirarAutorizacao(AutorizacaoId id, MotivoExpiracao motivo) { ... }
}

```

Nomes genéricos proibidos: Handler, Manager, Helper, Util, Data, Process, Info, Common, Base. Use nomes de domínio.

Imutabilidade

[❌ Código Não Aderente]:

```

// classe com setters publicos: estado mutavel depois da construcao
public class Market {
    private Long id;
    private String name;
    public void setId(Long id) { this.id = id; }
}

```

```
    public void setName(String name) { this.name = name; }
}
```

[🔥 **Violação e Explicação**]: setters públicos expõem o estado interno mutável; o chamador pode alterar `id/name` após a construção, quebrando invariantes do domínio e criando bugs sutis em fluxos assíncronos.

[✅ **Exemplo de Refatoração**]:

```
// record para DTOs e value objects (imutavel, equals/hashCode/toString gerados)
public record MarketDto(Long id, String name, MarketStatus status) {}

// ou classe com final fields e getters only
public class Market {
    private final Long id;
    private final String name;
    // getters only, no setters
}
```

Optional — uso correto

[❌ **Código Não Aderente**]:

```
public Market buscarPorSlug(String slug) {
    Optional<Market> market = marketRepository.findBySlug(slug);
    return market.get(); // NoSuchElementException se vazio
}
```

[🔥 **Violação e Explicação**]: `Optional.get()` sem `.orElse().orElseThrow().isPresent()` joga a decisão para erro em runtime sem chance do chamador reagir.

[✅ **Exemplo de Refatoração**]:

```
public Market buscarPorSlug(String slug) {
    return marketRepository.findBySlug(slug)
        .orElseThrow(() -> new EntityNotFoundException("Market not found: " + slug));
}
```

Streams — pipelines curtos, sem efeito colateral

[❌ **Código Não Aderente**]:

```
List<String> nomesAtivos = new ArrayList<>();
markets.stream().forEach(m -> {
    if (m.isAtivo()) {
        nomesAtivos.add(m.name().toUpperCase());
    }
});
```

[🔥 **Violação e Explicação**]: `forEach` capturando variável externa é anti-pattern clássico; força rastrear o efeito colateral e impede paralelização.

[✅ **Exemplo de Refatoração**]:

```
List<String> names = markets.stream()
    .filter(Market::isAtivo)
    .map(m -> m.name().toUpperCase())
    .toList();
```

Exception handling

- Use **unchecked exceptions** para erros de domínio (`BusinessException`).
- Crie **exceções específicas** em vez de `RuntimeException` genérica.
- Evite `catch (Exception ex) amplo`.
- Sempre preserve a causa (`throw new ApplicationException(msg, e)`).

[❌ **Código Não Aderente**]:

```
try {
    integracaoClient.enviar(pedido);
} catch (IOException e) {
    throw new RuntimeException(e.getMessage()); // Perde a causa (e)
}
```

[🔥 **Violação e Explicação**]: perde a `Throwable` cause (stack trace) e a mensagem original. Mensagem vaga força a IA a gastar turnos extras para descobrir o que deu errado.

[✅ Exemplo de Refatoração]:

```
try {
    integracaoClient.enviar(pedido);
} catch (IOException e) {
    throw new ApplicationException("Falha ao enviar pedido " + pedido.id(), e);
}
```

Genéricos e type safety (Tipos explícitos para IA)**[❌ Código Não Aderente]:**

```
public Map indexById(Collection items) { ... } // sem type safety
```

[🔥 Violação e Explicação]: código sem anotações de tipo ou com raw types obriga agentes e humanos a inferirem o que entra e sai, gerando falhas. O agente poupa trabalho de descoberta em códigos tipados.

[✅ Exemplo de Refatoração]:

```
public <T extends Identifiable> Map<Long, T> indexById(Collection<T> items) { ... }
```

Tell, Don't Ask & No Getters/Setters (Object Calisthenics)

O código cliente não deve perguntar o estado interno de um objeto para tomar decisões. A regra "No Getters/Setters/Properties" foca no encapsulamento de comportamentos e evita a exposição indevida que viola a orientação a objetos. Uma classe exposta à manipulação de estado por fora gera domínio anêmico e separa os dados do comportamento.

[❌ Código Não Aderente]:

```
// Dominio anemico: o objeto e um saco de dados, a acao e feita de fora
public void aplicarDesconto(Produto produto) {
    if (produto.getPreco() > 50) {
        produto.setPreco(produto.getPreco() - 10);
    }
}
```

[🔥 Violação e Explicação]:

1. Ferimento de encapsulamento e do princípio "Tell, Don't Ask". O cliente pergunta o preço para decidir a regra.
2. O agente LLM precisa ler 2 arquivos (Produto + Service) para entender a regra.
3. Se a regra mudar, será necessário caçar onde esse getter/setter foi usado no código.

[✅ Exemplo de Refatoração]:

```
public class Produto {
    private BigDecimal preco;

    // O objeto controla e protege sua propria regra
    public void aplicarDesconto(BigDecimal valorDesconto) {
        if (this.preco.compareTo(new BigDecimal("50")) > 0) {
            this.preco = this.preco.subtract(valorDesconto);
        }
    }
}

// O cliente so envia o comando
public void processar(Produto produto) {
    produto.aplicarDesconto(new BigDecimal("10"));
}
```

Primitive Obsession & Wrap All Primitives (Object Calisthenics)

Não use tipos primitivos para representar conceitos com comportamento. A regra "Wrap All Primitives And Strings" determina que variáveis primitivas com comportamento específico de domínio (como validações próprias) devem ser encapsuladas em Objetos.

[❌ Código Não Aderente]:

```
public class Pessoa {
    private String cpf; // Obsessao por tipo primitivo

    public Pessoa(String cpf) {
        if (cpf == null || cpf.length() != 11) throw new IllegalArgumentException();
        this.cpf = cpf;
    }
}
```

[🔥 Violação e Explicação]: O CPF em formato String espalha lógica de validação. O agente de IA precisa inferir o formato, consumindo janela de contexto. A regra de validação não pertence estruturalmente à Pessoa.

[✅ Exemplo de Refatoração]:

```
public record Cpf(String numero) { // Value object imutavel que contem suas regras
    public Cpf {
        if (numero == null || numero.length() != 11) {
            throw new IllegalArgumentException("CPF Invalido");
        }
    }
}

public class Pessoa {
    private Cpf cpf; // Propriedade tipada e protegida
}
```

First Class Collections (Object Calisthenics)

Qualquer classe que contenha uma coleção não deve conter outras variáveis de membro. Se você tem um conjunto de elementos, crie uma classe dedicada exclusivamente a essa coleção.

[❌ Código Não Aderente]:

```
public class Empresa {
    private String razaoSocial;
    private List<Funcionario> funcionarios; // Colecao misturada com outros atributos

    // Metodos que manipulam a lista se misturam com metodos da empresa
    public List<Funcionario> buscarContadores() {
        return funcionarios.stream().filter(f -> f.getCargo().equals("contador")).toList();
    }
}
```

[🔥 Violação e Explicação]: Os comportamentos de filtro e agrupamento de funcionários poluem a classe Empresa.

[✅ Exemplo de Refatoração]:

```
public class QuadroFuncionarios { // First Class Collection
    private final List<Funcionario> funcionarios;

    public QuadroFuncionarios(List<Funcionario> funcionarios) {
        this.funcionarios = funcionarios;
    }

    // Comportamentos especificos tem um lar
    public List<Funcionario> buscarContadores() {
        return funcionarios.stream().filter(f -> f.getCargo().equals("contador")).toList();
    }
}

public class Empresa {
    private String razaoSocial;
    private QuadroFuncionarios quadro;
}
```

One Dot Per Line / Law of Demeter (Object Calisthenics)

Evite cadeias extensas de chamadas que atravessam vários objetos. Se você usa mais de um ponto na mesma linha, o seu objeto é um intermediário sabendo demais sobre a estrutura dos outros.

[❌ Código Não Aderente]:

```
// Navegando a estrutura interna (quebra de encapsulamento)
String nomeChefe = funcionario.getDepartamento().getChefe().getNome();
```

[🔥 Violação e Explicação]: A estrutura interna do objeto fica exposta, dificultando a leitura e acoplado fortemente as classes.

[✅ Exemplo de Refatoração]:

```
// O objeto expressa intencoes via metodos comportamentais diretos
String nomeChefe = funcionario.getNomeChefeDepartamento();
```

No Classes With More Than Two Instance Variables (Alta Coesão)

Classes não devem ter mais do que duas variáveis de instância, forçando um alto nível de coesão, composição e abstração. O objetivo prático é agrupar variáveis em lógicas menores quando a classe assume muitas responsabilidades.

[❌ Código Não Aderente]:

```
public class Funcionario {
    private String nome;
```

```
private int idade;
private String cargo;
private String departamento;
}
```

[🔥 **Violação e Explicação**]: A classe tem muitos atributos e assume informações tanto pessoais quanto contratuais.

[✅ **Exemplo de Refatoração**]:

```
public class Funcionario {
    private InformacoesPessoais dadosPessoais; // Agrupa nome e idade
    private InformacoesTrabalho dadosTrabalho; // Agrupa cargo e departamento
}
```

Replace Magic Number with Symbolic Constant

Qualquer literal numérico ou `String` com **significado de domínio** é proibido no meio de validações. Deve virar constante nomeada.

[❌ **Código Não Aderente**]:

```
if (valor.compareTo(new BigDecimal("50000")) > 0) { ... }
```

[🔥 **Violação e Explicação**]: Magic Numbers escondem a regra. O agente precisa adivinhar o motivo da limitação.

[✅ **Exemplo de Refatoração**]:

```
// Constante documenta a regra
private static final BigDecimal LIMITE_MAXIMO_PIX = new BigDecimal("50000");
if (valor.compareTo(LIMITE_MAXIMO_PIX) > 0) { ... }
```

Guard Clauses & Don't Use "Else" (Object Calisthenics)

O uso de `else` é **proibido**. Assuma o fluxo padrão e faça validações através de Fail-Fast, Early Return ou Guard Clauses. Cada nível de indentação extra multiplica o custo cognitivo.

[❌ **Código Não Aderente**]:

```
public void processar(Pedido pedido) {
    if (pedido != null) {
        if (!pedido.getItems().isEmpty()) {
            executar(pedido);
        } else {
            throw new BusinessException("sem itens");
        }
    }
}
```

[🔥 **Violação e Explicação**]: Aninhamento de ifs aumenta a complexidade e polui a legibilidade. Para LLMs, o "else" desvia a atenção da lógica contínua.

[✅ **Exemplo de Refatoração**]:

```
public void processar(Pedido pedido) {
    if (pedido == null) return;

    if (pedido.getItems().isEmpty()) {
        throw new BusinessException("sem itens");
    }

    executar(pedido);
}
```

Clean Code for AI (Arquitetura para o Agente)

Regras vitais para quando o LLM interage e edita a base de código:

- Only One Level Of Indentation Per Method & Manter Métodos Pequenos**: Métodos curtos, contendo de 4 a 20 linhas ou 1 nível de indentação, cabem na mesma tool call do LLM, evitando perda de contexto.
- SRP (Responsabilidade Única)**: Permite edições isoladas via AI sem efeito colateral destrutivo.
- Comentários de Proveniência**: Embora comentários óbvios (ex: `// incrementa i`) consumam tokens de IA à toa e devam sumir, documentações que explicam o *motivo* da decisão de negócio são cruciais.
- Testes que o agente consegue rodar**: Testes automatizados rápidos e headless (TDD) atuam como bússola, diferenciando o agente ágil do que trabalha "chutando".
- Injeção de Dependências**: Instanciar dependências hardcoded impede o isolamento no momento do teste. Usar DI facilita a refatoração e as suítes de teste.

Refactorings do Fowler e Bloaters — guia rápido

Além dos princípios, usamos técnicas do catálogo Refactoring Guru para sanar "Smells".

Remove Parameter

[❌ Código Não Aderente]:

```
// "isCloud" e recebido mas nunca influencia o resultado
public Backend selecionarBackend(long tableId, ConnectContext context, boolean isCloud) { ... }
```

[🚩 Violação e Explicação]: Parâmetro morto polui a assinatura e é uma abstração especulativa.

[✅ Exemplo de Refatoração]:

```
public Backend selecionarBackend(long tableId, ConnectContext context) { ... }
```

Extract Method

Resolvendo o problema do *Long Method* (Bloater).

[❌ Código Não Aderente]:

```
public void processar(Pedido pedido) {
    // 20 linhas de validacao aqui
    // logica principal
}
```

[🚩 Violação e Explicação]: Validações inline espessas dificultam ler o fluxo real de domínio. O agente perde o foco de qual ação principal é realizada.

[✅ Exemplo de Refatoração]:

```
public void processar(Pedido pedido) {
    validarPedido(pedido);
    // logica principal
}
```

Replace Conditional with Polymorphism

Resolvendo o cheiro de *Switch Statements* ou longos *ifs* encadeados que checam tipos.

[❌ Código Não Aderente]:

```
if (pagamento instanceof Pix) return BigDecimal.ZERO;
if (pagamento instanceof Cartao) return BigDecimal.valueOf(0.03);
throw new IllegalStateException("Tipo desconhecido");
```

[🚩 Violação e Explicação]: Não é extensível, é aberto a falhas em runtime ao adicionar novo tipo.

[✅ Exemplo de Refatoração]:

```
// Pattern matching / switch exaustivo moderno
return switch (pagamento) {
    case Pix p      -> BigDecimal.ZERO;
    case Cartao c   -> BigDecimal.valueOf(0.03);
};
```

Introduce Parameter Object

Trata *Data Clumps* onde o mesmo conjunto de parâmetros é copiado entre métodos.

[❌ Código Não Aderente]:

```
public void buscar(LocalDate inicio, LocalDate fim, String status, int pagina) { ... }
```

[🚩 Violação e Explicação]: Propensão a erros ao alterar assinatura; dados andam juntos mas não formam um objeto.

[✅ Exemplo de Refatoração]:

```
public record FiltroPedido(LocalDate inicio, LocalDate fim, String status) {}
public void buscar(FiltroPedido filtro, int pagina) { ... }
```

Quem aplica o quê

Situação	Quem	Skill
Aplicar refactoring em uma classe/método	sessão principal	esta skill
Revisar diff/PR com checklist de severidade	agent java-revisor	revisao-de-codigo-java
Remoção de parâmetro focada (passo-a-passo)	sessão principal	refactoring-remove-parameter
Limpar imports não usados	sessão principal	remover-imports-nao-usados
Centralizar configuração dispersa (Shotgun Surgery)	session/engenheiro-devops	java-architecture
Decidir onde mora um value object novo	session/java-construtor	arquitetura-limpa-java

User prompt: junte todos os trechos da nova skill em um arquivo skill.md e me permita fazer o download desse arquivo montado

Response: [🚩 **Violação e Explicação**]: mesma validação em 2 lugares — a 3ª ocorrência (em importarEmLote, por exemplo) confirma o padrão. Manter a duplicação significa N lugares para corrigir quando a regra mudar.

[✅ **Exemplo de Refatoração**]:

```
// fonte unica: metodo privado resolve sem criar interface/factory para o futuro
public class UsuarioService {
    public void criarUsuario(UsuarioRequest req) { validarEmail(req.getEmail()); /* ... */ }
    public void atualizarUsuario(UsuarioRequest req) { validarEmail(req.getEmail()); /* ... */ }

    private void validarEmail(String email) {
        if (email == null || !email.contains("@")) {
            throw new ValidationException("Email invalido");
        }
    }
}
```

KISS — Keep It Simple / YAGNI — You Aren't Gonna Need It

[❌ **Código Não Aderente**]:

```
// sobre-engenharia para 1 implementacao, sem segunda variacao a vista
public interface UserFactory {
    User createUser();
}
public class ConcreteUserFactory implements UserFactory {
    public User createUser() { return new User(); }
}
```

[🚩 **Violação e Explicação**]: interface + implementação única **"para o futuro"** é a abstração especulativa clássica (YAGNI). O custo (mais arquivos para ler, mais para o agente raciocinar) não traz benefício enquanto houver 1 variante.

[✅ **Exemplo de Refatoração**]:

```
// chamada direta; implemente a abstracao quando a segunda variacao aparecer de fato
public User createUser() { return new User(); }
```

Convenções de nomenclatura (Object Calisthenics: Don't Abbreviate)

A regra do Object Calisthenics orienta a nunca usar nomes abreviados, pois nomes com significado ajudam no entendimento e previnem falhas de design.

```
// Classes/Records: PascalCase
public class MarketService {}
public record Money(BigDecimal amount, Currency currency) {}

// Metodos/campos: camelCase
private final MarketRepository marketRepository;
public Market findBySlug(String slug) {}

// Constantes: UPPER_SNAKE_CASE
private static final int MAX_PAGE_SIZE = 100;
```

Nomes que revelam intenção e são grepáveis:

[❌ **Código Não Aderente**]:

```
// abreviacoes obscuras e nomes genericos nao sao grepaveis
public List<Produto> get(String s) { ... }
public boolean chk(String str) { ... }
```



```
private static final int N = 100;
public class Handler { public void handle(String p, String rng) { ... } }
```

[🔥 Violação e Explicação]: nomes genéricos (Handler, get, N, chk, p, rng) poluem a busca lexical, escondem a intenção e violam a regra "Don't Abbreviate". O agente tem que ler o corpo para descobrir o que o método faz. Se pesquisar pelo nome retorna coisas irrelevantes, o nome está ruim para a IA.

[✅ Exemplo de Refatoração]:

```
// nome diz o que faz; busca lexical (rg "AutorizacaoExpiradaHandler") cai direto
public List<Produto> buscarAtivosPorCategoria(String categoria) { ... }
public boolean precoEhValido(BigDecimal preco) { ... }
private static final int TAMANHO_MAXIMO_PAGINA = 100;
public class AutorizacaoExpiradaHandler {
    public void expirarAutorizacao(AutorizacaoId id, MotivoExpiracao motivo) { ... }
}
```

Nomes genéricos proibidos: Handler, Manager, Helper, Util, Data, Process, Info, Common, Base. Use nomes de domínio.

Imutabilidade

[❌ Código Não Aderente]:

```
// classe com setters publicos: estado mutavel depois da construcao
public class Market {
    private Long id;
    private String name;
    public void setId(Long id) { this.id = id; }
    public void setName(String name) { this.name = name; }
}
```

[🔥 Violação e Explicação]: setters públicos expõem o estado interno mutável; o chamador pode alterar id/name após a construção, quebrando invariantes do domínio e criando bugs sutis em fluxos assíncronos.

[✅ Exemplo de Refatoração]:

```
// record para DTOs e value objects (imutavel, equals/hashCode/toString gerados)
public record MarketDto(Long id, String name, MarketStatus status) {}

// ou classe com final fields e getters only
public class Market {
    private final Long id;
    private final String name;
    // getters only, no setters
}
```

Optional — uso correto

[❌ Código Não Aderente]:

```
public Market buscarPorSlug(String slug) {
    Optional<Market> market = marketRepository.findBySlug(slug);
    return market.get(); // NoSuchElementException se vazio
}
```

[🔥 Violação e Explicação]: Optional.get() sem .orElse/.orElseThrow/.isPresent() joga a decisão para erro em runtime sem chance do chamador reagir.

[✅ Exemplo de Refatoração]:

```
public Market buscarPorSlug(String slug) {
    return marketRepository.findBySlug(slug)
        .orElseThrow(() -> new EntityNotFoundException("Market not found: " + slug));
}
```

Streams — pipelines curtos, sem efeito colateral

[❌ Código Não Aderente]:

```
List<String> nomesAtivos = new ArrayList<>();
markets.stream().forEach(m -> {
    if (m.isAtivo()) {
        nomesAtivos.add(m.name().toUpperCase());
    }
});
```

[🔥 Violação e Explicação]: forEach capturando variável externa é anti-pattern clássico; força rastrear o efeito colateral e impede paralelização.

[✓ Exemplo de Refatoração]:

```
List<String> names = markets.stream()
    .filter(Market::isAtivo)
    .map(m -> m.name().toUpperCase())
    .toList();
```

Exception handling

- Use **unchecked exceptions** para erros de domínio (`BusinessException`).
- **Crie exceções específicas** em vez de `RuntimeException` genérica.
- **Evite** `catch (Exception ex) amplo`.
- **Sempre preserve a causa** (`throw new ApplicationException(msg, e)`).

[✗ Código Não Aderente]:

```
try {
    integracaoClient.enviar(pedido);
} catch (IOException e) {
    throw new RuntimeException(e.getMessage()); // Perde a causa (e)
}
```

[🔥 Violação e Explicação]: perde a `Throwable` `cause` (stack trace) e a mensagem original. Mensagem vaga força a IA a gastar turnos extras para descobrir o que deu errado.

[✓ Exemplo de Refatoração]:

```
try {
    integracaoClient.enviar(pedido);
} catch (IOException e) {
    throw new ApplicationException("Falha ao enviar pedido " + pedido.id(), e);
}
```

Genéricos e type safety (Tipos explícitos para IA)**[✗ Código Não Aderente]:**

```
public Map indexById(Collection items) { ... } // sem type safety
```

[🔥 Violação e Explicação]: código sem anotações de tipo ou com raw types obriga agentes e humanos a inferirem o que entra e sai, gerando falhas. O agente poupa trabalho de descoberta em códigos tipados.

[✓ Exemplo de Refatoração]:

```
public <T extends Identifiable> Map<Long, T> indexById(Collection<T> items) { ... }
```

Tell, Don't Ask & No Getters/Setters (Object Calisthenics)

O código cliente não deve perguntar o estado interno de um objeto para tomar decisões. A regra "No Getters/Setters/Properties" foca no encapsulamento de comportamentos e evita a exposição indevida que viola a orientação a objetos. Uma classe exposta à manipulação de estado por fora gera domínio anêmico e separa os dados do comportamento.

[✗ Código Não Aderente]:

```
// Dominio anemico: o objeto e um saco de dados, a acao e feita de fora
public void aplicarDesconto(Produto produto) {
    if (produto.getPreco() > 50) {
        produto.setPreco(produto.getPreco() - 10);
    }
}
```

[🔥 Violação e Explicação]:

1. Ferimento de encapsulamento e do princípio "Tell, Don't Ask". O cliente pergunta o preço para decidir a regra.
2. O agente LLM precisa ler 2 arquivos (`Produto` + `Service`) para entender a regra.
3. Se a regra mudar, será necessário caçar onde esse getter/setter foi usado no código.

[✓ Exemplo de Refatoração]:

```
public class Produto {
    private BigDecimal preco;

    // O objeto controla e protege sua propria regra
    public void aplicarDesconto(BigDecimal valorDesconto) {
        if (this.preco.compareTo(new BigDecimal("50")) > 0) {
```

```

        this.preco = this.preco.subtract(valorDesconto);
    }
}

// O cliente so envia o comando
public void processar(Produto produto) {
    produto.aplicarDesconto(new BigDecimal("10"));
}

```

Primitive Obsession & Wrap All Primitives (Object Calisthenics)

Não use tipos primitivos para representar conceitos com comportamento. A regra "Wrap All Primitives And Strings" determina que variáveis primitivas com comportamento específico de domínio (como validações próprias) devem ser encapsuladas em Objetos.

[❌ Código Não Aderente]:

```

public class Pessoa {
    private String cpf; // Obsessao por tipo primitivo

    public Pessoa(String cpf) {
        if (cpf == null || cpf.length() != 11) throw new IllegalArgumentException();
        this.cpf = cpf;
    }
}

```

[🔥 **Violação e Explicação**]: O CPF em formato String espalha lógica de validação. O agente de IA precisa inferir o formato, consumindo janela de contexto. A regra de validação não pertence estruturalmente à Pessoa.

[✅ Exemplo de Refatoração]:

```

public record Cpf(String numero) { // Value object imutavel que contem suas regras
    public Cpf {
        if (numero == null || numero.length() != 11) {
            throw new IllegalArgumentException("CPF Invalido");
        }
    }
}

public class Pessoa {
    private Cpf cpf; // Propriedade tipada e protegida
}

```

First Class Collections (Object Calisthenics)

Qualquer classe que contenha uma coleção não deve conter outras variáveis de membro. Se você tem um conjunto de elementos, crie uma classe dedicada exclusivamente a essa coleção.

[❌ Código Não Aderente]:

```

public class Empresa {
    private String razaoSocial;
    private List<Funcionario> funcionarios; // Colecao misturada com outros atributos

    // Metodos que manipulam a lista se misturam com metodos da empresa
    public List<Funcionario> buscarContadores() {
        return funcionarios.stream().filter(f -> f.getCargo().equals("contador")).toList();
    }
}

```

[🔥 **Violação e Explicação**]: Os comportamentos de filtro e agrupamento de funcionários poluem a classe Empresa.

[✅ Exemplo de Refatoração]:

```

public class QuadroFuncionarios { // First Class Collection
    private final List<Funcionario> funcionarios;

    public QuadroFuncionarios(List<Funcionario> funcionarios) {
        this.funcionarios = funcionarios;
    }

    // Comportamentos especificos tem um lar
    public List<Funcionario> buscarContadores() {
        return funcionarios.stream().filter(f -> f.getCargo().equals("contador")).toList();
    }
}

public class Empresa {
    private String razaoSocial;
}

```

```
private QuadroFuncionarios quadro;  
}
```

One Dot Per Line / Law of Demeter (Object Calisthenics)

Evite cadeias extensas de chamadas que atravessam vários objetos. Se você usa mais de um ponto na mesma linha, o seu objeto é um intermediário sabendo demais sobre a estrutura dos outros.

[❌ Código Não Aderente]:

```
// Navegando a estrutura interna (quebra de encapsulamento)  
String nomeChefe = funcionario.getDepartamento().getChefe().getNome();
```

[🔥 Violação e Explicação]: A estrutura interna do objeto fica exposta, dificultando a leitura e acoplando fortemente as classes.

[✅ Exemplo de Refatoração]:

```
// O objeto expressa intencoes via metodos comportamentais diretos  
String nomeChefe = funcionario.getNomeChefeDepartamento();
```

No Classes With More Than Two Instance Variables (Alta Coesão)

Classes não devem ter mais do que duas variáveis de instância, forçando um alto nível de coesão, composição e abstração. O objetivo prático é agrupar variáveis em lógicas menores quando a classe assume muitas responsabilidades.

[❌ Código Não Aderente]:

```
public class Funcionario {  
    private String nome;  
    private int idade;  
    private String cargo;  
    private String departamento;  
}
```

[🔥 Violação e Explicação]: A classe tem muitos atributos e assume informações tanto pessoais quanto contratuais.

[✅ Exemplo de Refatoração]:

```
public class Funcionario {  
    private InformacoesPessoais dadosPessoais; // Agrupa nome e idade  
    private InformacoesTrabalho dadosTrabalho; // Agrupa cargo e departamento  
}
```

Replace Magic Number with Symbolic Constant

Qualquer literal numérico ou `String` com **significado de domínio** é proibido no meio de validações. Deve virar constante nomeada.

[❌ Código Não Aderente]:

```
if (valor.compareTo(new BigDecimal("50000")) > 0) { ... }
```

[🔥 Violação e Explicação]: Magic Numbers escondem a regra. O agente precisa adivinhar o motivo da limitação.

[✅ Exemplo de Refatoração]:

```
// Constante documenta a regra  
private static final BigDecimal LIMITE_MAXIMO_PIX = new BigDecimal("50000");  
if (valor.compareTo(LIMITE_MAXIMO_PIX) > 0) { ... }
```

Guard Clauses & Don't Use "Else" (Object Calisthenics)

O uso de `else` é **proibido**. Assuma o fluxo padrão e faça validações através de Fail-Fast, Early Return ou Guard Clauses. Cada nível de indentação extra multiplica o custo cognitivo.

[❌ Código Não Aderente]:

```
public void processar(Pedido pedido) {  
    if (pedido != null) {  
        if (!pedido.getItems().isEmpty()) {  
            executar(pedido);  
        } else {  
            throw new BusinessException("sem itens");  
        }  
    }  
}
```

[🔥 Violação e Explicação]: Aninhamento de ifs aumenta a complexidade e polui a legibilidade. Para LLMs, o "else" desvia a atenção da lógica contínua.

[✓ Exemplo de Refatoração]:

```
public void processar(Pedido pedido) {
    if (pedido == null) return;

    if (pedido.getItems().isEmpty()) {
        throw new BusinessException("sem itens");
    }

    executar(pedido);
}
```

Clean Code for AI (Arquitetura para o Agente)

Regras vitais para quando o LLM interage e edita a base de código:

1. **Only One Level Of Indentation Per Method & Manter Métodos Pequenos:** Métodos curtos, contendo de 4 a 20 linhas ou 1 nível de indentação, cabem na mesma tool call do LLM, evitando perda de contexto.
2. **SRP (Responsabilidade Única):** Permite edições isoladas via AI sem efeito colateral destrutivo.
3. **Comentários de Proveniência:** Embora comentários óbvios (ex: `// incrementa i`) consumam tokens de IA à toa e devam sumir, documentações que explicam o *motivo* da decisão de negócio são cruciais.
4. **Testes que o agente consegue rodar:** Testes automatizados rápidos e headless (TDD) atuam como bússola, diferenciando o agente ágil do que trabalha "chutando".
5. **Injeção de Dependências:** Instanciar dependências hardcoded impede o isolamento no momento do teste. Usar DI facilita a refatoração e as suítes de teste.

Refactorings do Fowler e Bloaters — guia rápido

Além dos princípios, usamos técnicas do catálogo Refactoring Guru para sanar "Smells".

Remove Parameter**[✗ Código Não Aderente]:**

```
// "isCloud" e recebido mas nunca influencia o resultado
public Backend selecionarBackend(long tableId, ConnectContext context, boolean isCloud) { ... }
```

[🔥 Violação e Explicação]: Parâmetro morto polui a assinatura e é uma abstração especulativa.

[✓ Exemplo de Refatoração]:

```
public Backend selecionarBackend(long tableId, ConnectContext context) { ... }
```

Extract Method

Resolvendo o problema do *Long Method* (Bloater).

[✗ Código Não Aderente]:

```
public void processar(Pedido pedido) {
    // 20 linhas de validacao aqui
    // logica principal
}
```

[🔥 Violação e Explicação]: Validações inline espessas dificultam ler o fluxo real de domínio. O agente perde o foco de qual ação principal é realizada.

[✓ Exemplo de Refatoração]:

```
public void processar(Pedido pedido) {
    validarPedido(pedido);
    // logica principal
}
```

Replace Conditional with Polymorphism

Resolvendo o cheiro de *Switch Statements* ou longos *ifs* encadeados que checam tipos.

[✗ Código Não Aderente]:

```
if (pagamento instanceof Pix) return BigDecimal.ZERO;
if (pagamento instanceof Cartao) return BigDecimal.valueOf(0.03);
throw new IllegalStateException("Tipo desconhecido");
```

[🔥 **Violação e Explicação**]: Não é extensível, é aberto a falhas em runtime ao adicionar novo tipo.

[✅ **Exemplo de Refatoração**]:

```
// Pattern matching / switch exaustivo moderno
return switch (pagamento) {
    case Pix p      -> BigDecimal.ZERO;
    case Cartao c   -> BigDecimal.valueOf(0.03);
};
```

Introduce Parameter Object

Trata *Data Clumps* onde o mesmo conjunto de parâmetros é copiado entre métodos.

[❌ **Código Não Aderente**]:

```
public void buscar(LocalDate inicio, LocalDate fim, String status, int pagina) { ... }
```

[🔥 **Violação e Explicação**]: Propensão a erros ao alterar assinatura; dados andam juntos mas não formam um objeto.

[✅ **Exemplo de Refatoração**]:

```
public record FiltroPedido(LocalDate inicio, LocalDate fim, String status) {}
public void buscar(FiltroPedido filtro, int pagina) { ... }
```

Quem aplica o quê

Situação	Quem	Skill
Aplicar refactoring em uma classe/método	sessão principal	esta skill
Revisar diff/PR com checklist de severidade	agent java-revisor	revisao-de-codigo-java
Remoção de parâmetro focada (passo-a-passo)	sessão principal	refactoring-remove-parameter
Limpar imports não usados	sessão principal	remover-imports-nao-usados
Centralizar configuração dispersa (Shotgun Surgery)	session/engenheiro-devops	java-architecture
Decidir onde mora um value object novo	session/java-construtor	arquitetura-limpa-java

```
with open("SKILL_v1.2.0.md", "w", encoding="utf-8") as f: f.write(content)

print(f"File created successfully at {os.path.abspath('SKILL_v1.2.0.md')}")
```

```
```python?code_reference&code_event_index=2
import os

content = """---
name: qualidade-codigo-java
description: "Application-side guide for clean code in Java – DRY, KISS, YAGNI, naming, immutability, `Optional`, st
license: MIT
metadata:
 author: https://github.com/srportto/srportto
 co-author: https://github.com/Jeffallan/claude-skills
 version: "1.2.0"
 domain: code-quality
 triggers: clean code, boas práticas, refatorar, DRY, KISS, YAGNI, imutabilidade, Optional, streams, Fowler, Object
 role: reference
 scope: code-quality
 output-format: code
 related-skills: revisao-de-codigo-java, padroes-de-projeto-java, refatorador-java, java-moderno

Qualidade de Código Java (clean code + refactoring)

Visão geral

Guia de **aplicação** de clean code em Java – DRY, KISS, YAGNI, nomenclatura, imutabilidade, `Optional`, streams, ex

Quando NÃO usar: para revisar um diff/PR com checklist por severidade, use `revisao-de-codigo-java` (ela referen

Clean code – princípios com exemplo
```

> **Coesão com `revisao-de-codigo-java`:** esta skill é o "lado ativo" (o **como** aplicar cada refactoring). A `rev`

> **Princípio-mestre (Clean Code for AI):** além de bom para humanos, todo código deste catálogo deve estar **otimiz**

### DRY — Don't Repeat Yourself

**[❌ Código Não Aderente]:**

```
```java
// logica de validacao duplicada em dois metodos
public void criarUsuario(UsuarioRequest req) {
    if (req.getEmail() == null || !req.getEmail().contains("@")) {
        throw new ValidationException("Email invalido");
    }
}

public void atualizarUsuario(UsuarioRequest req) {
    if (req.getEmail() == null || !req.getEmail().contains("@")) {
        throw new ValidationException("Email invalido");
    }
}
```

[🔥 Violação e Explicação]: mesma validação em 2 lugares — a 3ª ocorrência (em `importarEmLote`, por exemplo) confirma o padrão. Manter a duplicação significa N lugares para corrigir quando a regra mudar.

[✅ Exemplo de Refatoração]:

```
// fonte unica: metodo privado resolve sem criar interface/factory para o futuro
public class UsuarioService {
    public void criarUsuario(UsuarioRequest req) { validarEmail(req.getEmail()); /* ... */ }
    public void atualizarUsuario(UsuarioRequest req) { validarEmail(req.getEmail()); /* ... */ }

    private void validarEmail(String email) {
        if (email == null || !email.contains("@")) {
            throw new ValidationException("Email invalido");
        }
    }
}
```

KISS — Keep It Simple / YAGNI — You Aren't Gonna Need It

[❌ Código Não Aderente]:

```
// sobre-engenharia para 1 implementacao, sem segunda variacao a vista
public interface UserFactory {
    User createUser();
}

public class ConcreteUserFactory implements UserFactory {
    public User createUser() { return new User(); }
}
```

[🔥 Violação e Explicação]: interface + implementação única **"para o futuro"** é a abstração especulativa clássica (YAGNI). O custo (mais arquivos para ler, mais para o agente raciocinar) não traz benefício enquanto houver 1 variante.

[✅ Exemplo de Refatoração]:

```
// chamada direta; implemente a abstracao quando a segunda variacao aparecer de fato
public User createUser() { return new User(); }
```

Convenções de nomenclatura (Object Calisthenics: Don't Abbreviate)

A regra do Object Calisthenics orienta a nunca usar nomes abreviados, pois nomes com significado ajudam no entendimento e previnem falhas de design.

```
// Classes/Records: PascalCase
public class MarketService {}
public record Money(BigDecimal amount, Currency currency) {}

// Metodos/campos: camelCase
private final MarketRepository marketRepository;
public Market findBySlug(String slug) {}

// Constantes: UPPER_SNAKE_CASE
private static final int MAX_PAGE_SIZE = 100;
```

Nomes que revelam intenção e são grepáveis:

[❌ Código Não Aderente]:

```
// abreviacoas obscuras e nomes genericos nao sao grepaveis
public List<Produto> get(String s) { ... }
```

```
public boolean chk(String str) { ... }
private static final int N = 100;
public class Handler { public void handle(String p, String rng) { ... } }
```

[🚩 Violação e Explicação]: nomes genéricos (Handler, get, N, chk, p, rng) poluem a busca lexical, escondem a intenção e violam a regra "Don't Abbreviate". O agente tem que ler o corpo para descobrir o que o método faz. Se pesquisar pelo nome retorna coisas irrelevantes, o nome está ruim para a IA.

[✅ Exemplo de Refatoração]:

```
// nome diz o que faz; busca lexical (rg "AutorizacaoExpiradaHandler") cai direto
public List<Produto> buscarAtivosPorCategoria(String categoria) { ... }
public boolean precoEhValido(BigDecimal preco) { ... }
private static final int TAMANHO_MAXIMO_PAGINA = 100;
public class AutorizacaoExpiradaHandler {
    public void expirarAutorizacao(AutorizacaoId id, MotivoExpiracao motivo) { ... }
}
```

Nomes genéricos proibidos: Handler, Manager, Helper, Util, Data, Process, Info, Common, Base. Use nomes de domínio.

Imutabilidade

[❌ Código Não Aderente]:

```
// classe com setters publicos: estado mutavel depois da construcao
public class Market {
    private Long id;
    private String name;
    public void setId(Long id) { this.id = id; }
    public void setName(String name) { this.name = name; }
}
```

[🚩 Violação e Explicação]: setters públicos expõem o estado interno mutável; o chamador pode alterar id/name após a construção, quebrando invariantes do domínio e criando bugs sutis em fluxos assíncronos.

[✅ Exemplo de Refatoração]:

```
// record para DTOs e value objects (imutavel, equals/hashCode/toString gerados)
public record MarketDto(Long id, String name, MarketStatus status) {}

// ou classe com final fields e getters only
public class Market {
    private final Long id;
    private final String name;
    // getters only, no setters
}
```

Optional — uso correto

[❌ Código Não Aderente]:

```
public Market buscarPorSlug(String slug) {
    Optional<Market> market = marketRepository.findBySlug(slug);
    return market.get(); // NoSuchElementException se vazio
}
```

[🚩 Violação e Explicação]: Optional.get() sem .orElse().orElseThrow().isPresent() joga a decisão para erro em runtime sem chance do chamador reagir.

[✅ Exemplo de Refatoração]:

```
public Market buscarPorSlug(String slug) {
    return marketRepository.findBySlug(slug)
        .orElseThrow(() -> new EntityNotFoundException("Market not found: " + slug));
}
```

Streams — pipelines curtos, sem efeito colateral

[❌ Código Não Aderente]:

```
List<String> nomesAtivos = new ArrayList<>();
markets.stream().forEach(m -> {
    if (m.isAtivo()) {
        nomesAtivos.add(m.name().toUpperCase());
    }
});
```

[🚩 Violação e Explicação]: forEach capturando variável externa é anti-pattern clássico; força rastrear o efeito colateral e impede paralelização.

[✓ Exemplo de Refatoração]:

```
List<String> names = markets.stream()
    .filter(Market::isAtivo)
    .map(m -> m.name().toUpperCase())
    .toList();
```

Exception handling

- Use **unchecked exceptions** para erros de domínio (`BusinessException`).
- **Crie exceções específicas** em vez de `RuntimeException` genérica.
- **Evite** `catch (Exception ex)` amplo.
- **Sempre preserve a causa** (`throw new ApplicationException(msg, e)`).

[✗ Código Não Aderente]:

```
try {
    integracaoClient.enviar(pedido);
} catch (IOException e) {
    throw new RuntimeException(e.getMessage()); // Perde a causa (e)
}
```

[🔥 Violação e Explicação]: perde a `Throwable` `cause` (stack trace) e a mensagem original. Mensagem vaga força a IA a gastar turnos extras para descobrir o que deu errado.

[✓ Exemplo de Refatoração]:

```
try {
    integracaoClient.enviar(pedido);
} catch (IOException e) {
    throw new ApplicationException("Falha ao enviar pedido " + pedido.id(), e);
}
```

Genéricos e type safety (Tipos explícitos para IA)**[✗ Código Não Aderente]:**

```
public Map indexById(Collection items) { ... } // sem type safety
```

[🔥 Violação e Explicação]: código sem anotações de tipo ou com raw types obriga agentes e humanos a inferirem o que entra e sai, gerando falhas. O agente poupa trabalho de descoberta em códigos tipados.

[✓ Exemplo de Refatoração]:

```
public <T extends Identifiable> Map<Long, T> indexById(Collection<T> items) { ... }
```

Tell, Don't Ask & No Getters/Setters (Object Calisthenics)

O código cliente não deve perguntar o estado interno de um objeto para tomar decisões. A regra "No Getters/Setters/Properties" foca no encapsulamento de comportamentos e evita a exposição indevida que viola a orientação a objetos. Uma classe exposta à manipulação de estado por fora gera domínio anêmico e separa os dados do comportamento.

[✗ Código Não Aderente]:

```
// Dominio anemico: o objeto e um saco de dados, a acao e feita de fora
public void aplicarDesconto(Produto produto) {
    if (produto.getPreco() > 50) {
        produto.setPreco(produto.getPreco() - 10);
    }
}
```

[🔥 Violação e Explicação]:

1. Ferimento de encapsulamento e do princípio "Tell, Don't Ask". O cliente pergunta o preço para decidir a regra.
2. O agente LLM precisa ler 2 arquivos (`Produto` + `Service`) para entender a regra.
3. Se a regra mudar, será necessário caçar onde esse getter/setter foi usado no código.

[✓ Exemplo de Refatoração]:

```
public class Produto {
    private BigDecimal preco;

    // O objeto controla e protege sua propria regra
    public void aplicarDesconto(BigDecimal valorDesconto) {
        if (this.preco.compareTo(new BigDecimal("50")) > 0) {
```

```

        this.preco = this.preco.subtract(valorDesconto);
    }
}

// O cliente so envia o comando
public void processar(Produto produto) {
    produto.aplicarDesconto(new BigDecimal("10"));
}

```

Primitive Obsession & Wrap All Primitives (Object Calisthenics)

Não use tipos primitivos para representar conceitos com comportamento. A regra "Wrap All Primitives And Strings" determina que variáveis primitivas com comportamento específico de domínio (como validações próprias) devem ser encapsuladas em Objetos.

[❌ Código Não Aderente]:

```

public class Pessoa {
    private String cpf; // Obsessao por tipo primitivo

    public Pessoa(String cpf) {
        if (cpf == null || cpf.length() != 11) throw new IllegalArgumentException();
        this.cpf = cpf;
    }
}

```

[🔥 **Violação e Explicação**]: O CPF em formato String espalha lógica de validação. O agente de IA precisa inferir o formato, consumindo janela de contexto. A regra de validação não pertence estruturalmente à Pessoa.

[✅ Exemplo de Refatoração]:

```

public record Cpf(String numero) { // Value object imutavel que contem suas regras
    public Cpf {
        if (numero == null || numero.length() != 11) {
            throw new IllegalArgumentException("CPF Invalido");
        }
    }
}

public class Pessoa {
    private Cpf cpf; // Propriedade tipada e protegida
}

```

First Class Collections (Object Calisthenics)

Qualquer classe que contenha uma coleção não deve conter outras variáveis de membro. Se você tem um conjunto de elementos, crie uma classe dedicada exclusivamente a essa coleção.

[❌ Código Não Aderente]:

```

public class Empresa {
    private String razaoSocial;
    private List<Funcionario> funcionarios; // Colecao misturada com outros atributos

    // Metodos que manipulam a lista se misturam com metodos da empresa
    public List<Funcionario> buscarContadores() {
        return funcionarios.stream().filter(f -> f.getCargo().equals("contador")).toList();
    }
}

```

[🔥 **Violação e Explicação**]: Os comportamentos de filtro e agrupamento de funcionários poluem a classe Empresa.

[✅ Exemplo de Refatoração]:

```

public class QuadroFuncionarios { // First Class Collection
    private final List<Funcionario> funcionarios;

    public QuadroFuncionarios(List<Funcionario> funcionarios) {
        this.funcionarios = funcionarios;
    }

    // Comportamentos especificos tem um lar
    public List<Funcionario> buscarContadores() {
        return funcionarios.stream().filter(f -> f.getCargo().equals("contador")).toList();
    }
}

public class Empresa {
    private String razaoSocial;
}

```

```
private QuadroFuncionarios quadro;  
}
```

One Dot Per Line / Law of Demeter (Object Calisthenics)

Evite cadeias extensas de chamadas que atravessam vários objetos. Se você usa mais de um ponto na mesma linha, o seu objeto é um intermediário sabendo demais sobre a estrutura dos outros.

[❌ Código Não Aderente]:

```
// Navegando a estrutura interna (quebra de encapsulamento)  
String nomeChefe = funcionario.getDepartamento().getChefe().getNome();
```

[🔥 Violação e Explicação]: A estrutura interna do objeto fica exposta, dificultando a leitura e acoplando fortemente as classes.

[✅ Exemplo de Refatoração]:

```
// O objeto expressa intencoes via metodos comportamentais diretos  
String nomeChefe = funcionario.getNomeChefeDepartamento();
```

No Classes With More Than Two Instance Variables (Alta Coesão)

Classes não devem ter mais do que duas variáveis de instância, forçando um alto nível de coesão, composição e abstração. O objetivo prático é agrupar variáveis em lógicas menores quando a classe assume muitas responsabilidades.

[❌ Código Não Aderente]:

```
public class Funcionario {  
    private String nome;  
    private int idade;  
    private String cargo;  
    private String departamento;  
}
```

[🔥 Violação e Explicação]: A classe tem muitos atributos e assume informações tanto pessoais quanto contratuais.

[✅ Exemplo de Refatoração]:

```
public class Funcionario {  
    private InformacoesPessoais dadosPessoais; // Agrupa nome e idade  
    private InformacoesTrabalho dadosTrabalho; // Agrupa cargo e departamento  
}
```

Replace Magic Number with Symbolic Constant

Qualquer literal numérico ou `String` com **significado de domínio** é proibido no meio de validações. Deve virar constante nomeada.

[❌ Código Não Aderente]:

```
if (valor.compareTo(new BigDecimal("50000")) > 0) { ... }
```

[🔥 Violação e Explicação]: Magic Numbers escondem a regra. O agente precisa adivinhar o motivo da limitação.

[✅ Exemplo de Refatoração]:

```
// Constante documenta a regra  
private static final BigDecimal LIMITE_MAXIMO_PIX = new BigDecimal("50000");  
if (valor.compareTo(LIMITE_MAXIMO_PIX) > 0) { ... }
```

Guard Clauses & Don't Use "Else" (Object Calisthenics)

O uso de `else` é **proibido**. Assuma o fluxo padrão e faça validações através de Fail-Fast, Early Return ou Guard Clauses. Cada nível de indentação extra multiplica o custo cognitivo.

[❌ Código Não Aderente]:

```
public void processar(Pedido pedido) {  
    if (pedido != null) {  
        if (!pedido.getItems().isEmpty()) {  
            executar(pedido);  
        } else {  
            throw new BusinessException("sem itens");  
        }  
    }  
}
```

[🔥 Violação e Explicação]: Aninhamento de ifs aumenta a complexidade e polui a legibilidade. Para LLMs, o "else" desvia a atenção da lógica contínua.

[✓ Exemplo de Refatoração]:

```
public void processar(Pedido pedido) {
    if (pedido == null) return;

    if (pedido.getItems().isEmpty()) {
        throw new BusinessException("sem itens");
    }

    executar(pedido);
}
```

Clean Code for AI (Arquitetura para o Agente)

Regras vitais para quando o LLM interage e edita a base de código:

1. **Only One Level Of Indentation Per Method & Manter Métodos Pequenos:** Métodos curtos, contendo de 4 a 20 linhas ou 1 nível de indentação, cabem na mesma tool call do LLM, evitando perda de contexto.
2. **SRP (Responsabilidade Única):** Permite edições isoladas via AI sem efeito colateral destrutivo.
3. **Comentários de Proveniência:** Embora comentários óbvios (ex: `// incrementa i`) consumam tokens de IA à toa e devam sumir, documentações que explicam o *motivo* da decisão de negócio são cruciais.
4. **Testes que o agente consegue rodar:** Testes automatizados rápidos e headless (TDD) atuam como bússola, diferenciando o agente ágil do que trabalha "chutando".
5. **Injeção de Dependências:** Instanciar dependências hardcoded impede o isolamento no momento do teste. Usar DI facilita a refatoração e as suítes de teste.

Refactorings do Fowler e Bloaters — guia rápido

Além dos princípios, usamos técnicas do catálogo Refactoring Guru para sanar "Smells".

Remove Parameter**[✗ Código Não Aderente]:**

```
// "isCloud" e recebido mas nunca influencia o resultado
public Backend selecionarBackend(long tableId, ConnectContext context, boolean isCloud) { ... }
```

[🔥 Violação e Explicação]: Parâmetro morto polui a assinatura e é uma abstração especulativa.

[✓ Exemplo de Refatoração]:

```
public Backend selecionarBackend(long tableId, ConnectContext context) { ... }
```

Extract Method

Resolvendo o problema do *Long Method* (Bloater).

[✗ Código Não Aderente]:

```
public void processar(Pedido pedido) {
    // 20 linhas de validacao aqui
    // logica principal
}
```

[🔥 Violação e Explicação]: Validações inline espessas dificultam ler o fluxo real de domínio. O agente perde o foco de qual ação principal é realizada.

[✓ Exemplo de Refatoração]:

```
public void processar(Pedido pedido) {
    validarPedido(pedido);
    // logica principal
}
```

Replace Conditional with Polymorphism

Resolvendo o cheiro de *Switch Statements* ou longos *ifs* encadeados que checam tipos.

[✗ Código Não Aderente]:

```
if (pagamento instanceof Pix) return BigDecimal.ZERO;
if (pagamento instanceof Cartao) return BigDecimal.valueOf(0.03);
throw new IllegalStateException("Tipo desconhecido");
```

[🔥 **Violação e Explicação**]: Não é extensível, é aberto a falhas em runtime ao adicionar novo tipo.

[✅ **Exemplo de Refatoração**]:

```
// Pattern matching / switch exaustivo moderno
return switch (pagamento) {
    case Pix p      -> BigDecimal.ZERO;
    case Cartao c   -> BigDecimal.valueOf(0.03);
};
```

Introduce Parameter Object

Trata *Data Clumps* onde o mesmo conjunto de parâmetros é copiado entre métodos.

[❌ **Código Não Aderente**]:

```
public void buscar(LocalDate inicio, LocalDate fim, String status, int pagina) { ... }
```

[🔥 **Violação e Explicação**]: Propensão a erros ao alterar assinatura; dados andam juntos mas não formam um objeto.

[✅ **Exemplo de Refatoração**]:

```
public record FiltroPedido(LocalDate inicio, LocalDate fim, String status) {}
public void buscar(FiltroPedido filtro, int pagina) { ... }
```

Quem aplica o quê

Situação	Quem	Skill
Aplicar refactoring em uma classe/método	sessão principal	esta skill
Revisar diff/PR com checklist de severidade	agent java-revisor	revisao-de-codigo-java
Remoção de parâmetro focada (passo-a-passo)	sessão principal	refactoring-remove-parameter
Limpar imports não usados	sessão principal	remover-imports-nao-usados
Centralizar configuração dispersa (Shotgun Surgery)	session/engenheiro-devops	java-architecture
Decidir onde mora um value object novo	session/java-construtor	arquitetura-limpa-java

```
with open("/mnt/data/SKILL_v1.2.0.md", "w", encoding="utf-8") as f: f.write(content)

print(f"File created successfully at {os.path.abspath('/mnt/data/SKILL_v1.2.0.md')}")
```

```
```python?code_reference&code_event_index=3
import os

content = """---
name: qualidade-codigo-java
description: "Application-side guide for clean code in Java – DRY, KISS, YAGNI, naming, immutability, `Optional`, st
license: MIT
metadata:
 author: https://github.com/srportto/srportto
 co-author: https://github.com/Jeffallan/claude-skills
 version: "1.2.0"
 domain: code-quality
 triggers: clean code, boas práticas, refatorar, DRY, KISS, YAGNI, imutabilidade, Optional, streams, Fowler, Object
 role: reference
 scope: code-quality
 output-format: code
 related-skills: revisao-de-codigo-java, padroes-de-projeto-java, refatorador-java, java-moderno

Qualidade de Código Java (clean code + refactoring)

Visão geral

Guia de **aplicação** de clean code em Java – DRY, KISS, YAGNI, nomenclatura, imutabilidade, `Optional`, streams, ex

Quando NÃO usar: para revisar um diff/PR com checklist por severidade, use `revisao-de-codigo-java` (ela referen

Clean code – princípios com exemplo
```

> **Coesão com `revisao-de-codigo-java`:** esta skill é o "lado ativo" (o **como** aplicar cada refactoring). A `rev`

> **Princípio-mestre (Clean Code for AI):** além de bom para humanos, todo código deste catálogo deve estar **otimiz**

### DRY — Don't Repeat Yourself

**[❌ Código Não Aderente]:**

```
```java
// logica de validacao duplicada em dois metodos
public void criarUsuario(UsuarioRequest req) {
    if (req.getEmail() == null || !req.getEmail().contains("@")) {
        throw new ValidationException("Email invalido");
    }
}

public void atualizarUsuario(UsuarioRequest req) {
    if (req.getEmail() == null || !req.getEmail().contains("@")) {
        throw new ValidationException("Email invalido");
    }
}
```

[🔥 Violação e Explicação]: mesma validação em 2 lugares — a 3ª ocorrência (em `importarEmLote`, por exemplo) confirma o padrão. Manter a duplicação significa N lugares para corrigir quando a regra mudar.

[✅ Exemplo de Refatoração]:

```
// fonte unica: metodo privado resolve sem criar interface/factory para o futuro
public class UsuarioService {
    public void criarUsuario(UsuarioRequest req) { validarEmail(req.getEmail()); /* ... */ }
    public void atualizarUsuario(UsuarioRequest req) { validarEmail(req.getEmail()); /* ... */ }

    private void validarEmail(String email) {
        if (email == null || !email.contains("@")) {
            throw new ValidationException("Email invalido");
        }
    }
}
```

KISS — Keep It Simple / YAGNI — You Aren't Gonna Need It

[❌ Código Não Aderente]:

```
// sobre-engenharia para 1 implementacao, sem segunda variacao a vista
public interface UserFactory {
    User createUser();
}

public class ConcreteUserFactory implements UserFactory {
    public User createUser() { return new User(); }
}
```

[🔥 Violação e Explicação]: interface + implementação única **"para o futuro"** é a abstração especulativa clássica (YAGNI). O custo (mais arquivos para ler, mais para o agente raciocinar) não traz benefício enquanto houver 1 variante.

[✅ Exemplo de Refatoração]:

```
// chamada direta; implemente a abstracao quando a segunda variacao aparecer de fato
public User createUser() { return new User(); }
```

Convenções de nomenclatura (Object Calisthenics: Don't Abbreviate)

A regra do Object Calisthenics orienta a nunca usar nomes abreviados, pois nomes com significado ajudam no entendimento e previnem falhas de design.

```
// Classes/Records: PascalCase
public class MarketService {}
public record Money(BigDecimal amount, Currency currency) {}

// Metodos/campos: camelCase
private final MarketRepository marketRepository;
public Market findBySlug(String slug) {}

// Constantes: UPPER_SNAKE_CASE
private static final int MAX_PAGE_SIZE = 100;
```

Nomes que revelam intenção e são grepáveis:

[❌ Código Não Aderente]:

```
// abreviaco es obscuras e nomes genericos nao sao grepaveis
public List<Produto> get(String s) { ... }
```

```
public boolean chk(String str) { ... }
private static final int N = 100;
public class Handler { public void handle(String p, String rng) { ... } }
```

[🚩 Violação e Explicação]: nomes genéricos (Handler, get, N, chk, p, rng) poluem a busca lexical, escondem a intenção e violam a regra "Don't Abbreviate". O agente tem que ler o corpo para descobrir o que o método faz. Se pesquisar pelo nome retorna coisas irrelevantes, o nome está ruim para a IA.

[✅ Exemplo de Refatoração]:

```
// nome diz o que faz; busca lexical (rg "AutorizacaoExpiradaHandler") cai direto
public List<Produto> buscarAtivosPorCategoria(String categoria) { ... }
public boolean precoEhValido(BigDecimal preco) { ... }
private static final int TAMANHO_MAXIMO_PAGINA = 100;
public class AutorizacaoExpiradaHandler {
    public void expirarAutorizacao(AutorizacaoId id, MotivoExpiracao motivo) { ... }
}
```

Nomes genéricos proibidos: Handler, Manager, Helper, Util, Data, Process, Info, Common, Base. Use nomes de domínio.

Imutabilidade

[❌ Código Não Aderente]:

```
// classe com setters publicos: estado mutavel depois da construcao
public class Market {
    private Long id;
    private String name;
    public void setId(Long id) { this.id = id; }
    public void setName(String name) { this.name = name; }
}
```

[🚩 Violação e Explicação]: setters públicos expõem o estado interno mutável; o chamador pode alterar id/name após a construção, quebrando invariantes do domínio e criando bugs sutis em fluxos assíncronos.

[✅ Exemplo de Refatoração]:

```
// record para DTOs e value objects (imutavel, equals/hashCode/toString gerados)
public record MarketDto(Long id, String name, MarketStatus status) {}

// ou classe com final fields e getters only
public class Market {
    private final Long id;
    private final String name;
    // getters only, no setters
}
```

Optional — uso correto

[❌ Código Não Aderente]:

```
public Market buscarPorSlug(String slug) {
    Optional<Market> market = marketRepository.findBySlug(slug);
    return market.get(); // NoSuchElementException se vazio
}
```

[🚩 Violação e Explicação]: Optional.get() sem .orElse().orElseThrow().isPresent() joga a decisão para erro em runtime sem chance do chamador reagir.

[✅ Exemplo de Refatoração]:

```
public Market buscarPorSlug(String slug) {
    return marketRepository.findBySlug(slug)
        .orElseThrow(() -> new EntityNotFoundException("Market not found: " + slug));
}
```

Streams — pipelines curtos, sem efeito colateral

[❌ Código Não Aderente]:

```
List<String> nomesAtivos = new ArrayList<>();
markets.stream().forEach(m -> {
    if (m.isAtivo()) {
        nomesAtivos.add(m.name().toUpperCase());
    }
});
```

[🚩 Violação e Explicação]: forEach capturando variável externa é anti-pattern clássico; força rastrear o efeito colateral e impede paralelização.

[✓ Exemplo de Refatoração]:

```
List<String> names = markets.stream()
    .filter(Market::isAtivo)
    .map(m -> m.name().toUpperCase())
    .toList();
```

Exception handling

- Use **unchecked exceptions** para erros de domínio (`BusinessException`).
- **Crie exceções específicas** em vez de `RuntimeException` genérica.
- **Evite** `catch (Exception ex)` amplo.
- **Sempre preserve a causa** (`throw new ApplicationException(msg, e)`).

[✗ Código Não Aderente]:

```
try {
    integracaoClient.enviar(pedido);
} catch (IOException e) {
    throw new RuntimeException(e.getMessage()); // Perde a causa (e)
}
```

[🔥 Violação e Explicação]: perde a `Throwable` `cause` (stack trace) e a mensagem original. Mensagem vaga força a IA a gastar turnos extras para descobrir o que deu errado.

[✓ Exemplo de Refatoração]:

```
try {
    integracaoClient.enviar(pedido);
} catch (IOException e) {
    throw new ApplicationException("Falha ao enviar pedido " + pedido.id(), e);
}
```

Genéricos e type safety (Tipos explícitos para IA)**[✗ Código Não Aderente]:**

```
public Map indexById(Collection items) { ... } // sem type safety
```

[🔥 Violação e Explicação]: código sem anotações de tipo ou com raw types obriga agentes e humanos a inferirem o que entra e sai, gerando falhas. O agente poupa trabalho de descoberta em códigos tipados.

[✓ Exemplo de Refatoração]:

```
public <T extends Identifiable> Map<Long, T> indexById(Collection<T> items) { ... }
```

Tell, Don't Ask & No Getters/Setters (Object Calisthenics)

O código cliente não deve perguntar o estado interno de um objeto para tomar decisões. A regra "No Getters/Setters/Properties" foca no encapsulamento de comportamentos e evita a exposição indevida que viola a orientação a objetos. Uma classe exposta à manipulação de estado por fora gera domínio anêmico e separa os dados do comportamento.

[✗ Código Não Aderente]:

```
// Dominio anemico: o objeto e um saco de dados, a acao e feita de fora
public void aplicarDesconto(Produto produto) {
    if (produto.getPreco() > 50) {
        produto.setPreco(produto.getPreco() - 10);
    }
}
```

[🔥 Violação e Explicação]:

1. Ferimento de encapsulamento e do princípio "Tell, Don't Ask". O cliente pergunta o preço para decidir a regra.
2. O agente LLM precisa ler 2 arquivos (`Produto` + `Service`) para entender a regra.
3. Se a regra mudar, será necessário caçar onde esse getter/setter foi usado no código.

[✓ Exemplo de Refatoração]:

```
public class Produto {
    private BigDecimal preco;

    // O objeto controla e protege sua propria regra
    public void aplicarDesconto(BigDecimal valorDesconto) {
        if (this.preco.compareTo(new BigDecimal("50")) > 0) {
```



```

        this.preco = this.preco.subtract(valorDesconto);
    }
}

// O cliente so envia o comando
public void processar(Produto produto) {
    produto.aplicarDesconto(new BigDecimal("10"));
}

```

Primitive Obsession & Wrap All Primitives (Object Calisthenics)

Não use tipos primitivos para representar conceitos com comportamento. A regra "Wrap All Primitives And Strings" determina que variáveis primitivas com comportamento específico de domínio (como validações próprias) devem ser encapsuladas em Objetos.

[❌ Código Não Aderente]:

```

public class Pessoa {
    private String cpf; // Obsessao por tipo primitivo

    public Pessoa(String cpf) {
        if (cpf == null || cpf.length() != 11) throw new IllegalArgumentException();
        this.cpf = cpf;
    }
}

```

[🔥 **Violação e Explicação**]: O CPF em formato String espalha lógica de validação. O agente de IA precisa inferir o formato, consumindo janela de contexto. A regra de validação não pertence estruturalmente à Pessoa.

[✅ Exemplo de Refatoração]:

```

public record Cpf(String numero) { // Value object imutavel que contem suas regras
    public Cpf {
        if (numero == null || numero.length() != 11) {
            throw new IllegalArgumentException("CPF Invalido");
        }
    }
}

public class Pessoa {
    private Cpf cpf; // Propriedade tipada e protegida
}

```

First Class Collections (Object Calisthenics)

Qualquer classe que contenha uma coleção não deve conter outras variáveis de membro. Se você tem um conjunto de elementos, crie uma classe dedicada exclusivamente a essa coleção.

[❌ Código Não Aderente]:

```

public class Empresa {
    private String razaoSocial;
    private List<Funcionario> funcionarios; // Colecao misturada com outros atributos

    // Metodos que manipulam a lista se misturam com metodos da empresa
    public List<Funcionario> buscarContadores() {
        return funcionarios.stream().filter(f -> f.getCargo().equals("contador")).toList();
    }
}

```

[🔥 **Violação e Explicação**]: Os comportamentos de filtro e agrupamento de funcionários poluem a classe Empresa.

[✅ Exemplo de Refatoração]:

```

public class QuadroFuncionarios { // First Class Collection
    private final List<Funcionario> funcionarios;

    public QuadroFuncionarios(List<Funcionario> funcionarios) {
        this.funcionarios = funcionarios;
    }

    // Comportamentos especificos tem um lar
    public List<Funcionario> buscarContadores() {
        return funcionarios.stream().filter(f -> f.getCargo().equals("contador")).toList();
    }
}

public class Empresa {
    private String razaoSocial;
}

```

```
private QuadroFuncionarios quadro;  
}
```

One Dot Per Line / Law of Demeter (Object Calisthenics)

Evite cadeias extensas de chamadas que atravessam vários objetos. Se você usa mais de um ponto na mesma linha, o seu objeto é um intermediário sabendo demais sobre a estrutura dos outros.

[❌ Código Não Aderente]:

```
// Navegando a estrutura interna (quebra de encapsulamento)  
String nomeChefe = funcionario.getDepartamento().getChefe().getNome();
```

[🔥 Violação e Explicação]: A estrutura interna do objeto fica exposta, dificultando a leitura e acoplando fortemente as classes.

[✅ Exemplo de Refatoração]:

```
// O objeto expressa intencoes via metodos comportamentais diretos  
String nomeChefe = funcionario.getNomeChefeDepartamento();
```

No Classes With More Than Two Instance Variables (Alta Coesão)

Classes não devem ter mais do que duas variáveis de instância, forçando um alto nível de coesão, composição e abstração. O objetivo prático é agrupar variáveis em lógicas menores quando a classe assume muitas responsabilidades.

[❌ Código Não Aderente]:

```
public class Funcionario {  
    private String nome;  
    private int idade;  
    private String cargo;  
    private String departamento;  
}
```

[🔥 Violação e Explicação]: A classe tem muitos atributos e assume informações tanto pessoais quanto contratuais.

[✅ Exemplo de Refatoração]:

```
public class Funcionario {  
    private InformacoesPessoais dadosPessoais; // Agrupa nome e idade  
    private InformacoesTrabalho dadosTrabalho; // Agrupa cargo e departamento  
}
```

Replace Magic Number with Symbolic Constant

Qualquer literal numérico ou `String` com **significado de domínio** é proibido no meio de validações. Deve virar constante nomeada.

[❌ Código Não Aderente]:

```
if (valor.compareTo(new BigDecimal("50000")) > 0) { ... }
```

[🔥 Violação e Explicação]: Magic Numbers escondem a regra. O agente precisa adivinhar o motivo da limitação.

[✅ Exemplo de Refatoração]:

```
// Constante documenta a regra  
private static final BigDecimal LIMITE_MAXIMO_PIX = new BigDecimal("50000");  
if (valor.compareTo(LIMITE_MAXIMO_PIX) > 0) { ... }
```

Guard Clauses & Don't Use "Else" (Object Calisthenics)

O uso de `else` é **proibido**. Assuma o fluxo padrão e faça validações através de Fail-Fast, Early Return ou Guard Clauses. Cada nível de indentação extra multiplica o custo cognitivo.

[❌ Código Não Aderente]:

```
public void processar(Pedido pedido) {  
    if (pedido != null) {  
        if (!pedido.getItems().isEmpty()) {  
            executar(pedido);  
        } else {  
            throw new BusinessException("sem itens");  
        }  
    }  
}
```

[🔥 Violação e Explicação]: Aninhamento de ifs aumenta a complexidade e polui a legibilidade. Para LLMs, o "else" desvia a atenção da lógica contínua.

[✓ Exemplo de Refatoração]:

```
public void processar(Pedido pedido) {
    if (pedido == null) return;

    if (pedido.getItems().isEmpty()) {
        throw new BusinessException("sem itens");
    }

    executar(pedido);
}
```

Clean Code for AI (Arquitetura para o Agente)

Regras vitais para quando o LLM interage e edita a base de código:

1. **Only One Level Of Indentation Per Method & Manter Métodos Pequenos:** Métodos curtos, contendo de 4 a 20 linhas ou 1 nível de indentação, cabem na mesma tool call do LLM, evitando perda de contexto.
2. **SRP (Responsabilidade Única):** Permite edições isoladas via AI sem efeito colateral destrutivo.
3. **Comentários de Proveniência:** Embora comentários óbvios (ex: `// incrementa i`) consumam tokens de IA à toa e devam sumir, documentações que explicam o *motivo* da decisão de negócio são cruciais.
4. **Testes que o agente consegue rodar:** Testes automatizados rápidos e headless (TDD) atuam como bússola, diferenciando o agente ágil do que trabalha "chutando".
5. **Injeção de Dependências:** Instanciar dependências hardcoded impede o isolamento no momento do teste. Usar DI facilita a refatoração e as suítes de teste.

Refactorings do Fowler e Bloaters — guia rápido

Além dos princípios, usamos técnicas do catálogo Refactoring Guru para sanar "Smells".

Remove Parameter**[✗ Código Não Aderente]:**

```
// "isCloud" e recebido mas nunca influencia o resultado
public Backend selecionarBackend(long tableId, ConnectContext context, boolean isCloud) { ... }
```

[🔥 Violação e Explicação]: Parâmetro morto polui a assinatura e é uma abstração especulativa.

[✓ Exemplo de Refatoração]:

```
public Backend selecionarBackend(long tableId, ConnectContext context) { ... }
```

Extract Method

Resolvendo o problema do *Long Method* (Bloater).

[✗ Código Não Aderente]:

```
public void processar(Pedido pedido) {
    // 20 linhas de validacao aqui
    // logica principal
}
```

[🔥 Violação e Explicação]: Validações inline espessas dificultam ler o fluxo real de domínio. O agente perde o foco de qual ação principal é realizada.

[✓ Exemplo de Refatoração]:

```
public void processar(Pedido pedido) {
    validarPedido(pedido);
    // logica principal
}
```

Replace Conditional with Polymorphism

Resolvendo o cheiro de *Switch Statements* ou longos *ifs* encadeados que checam tipos.

[✗ Código Não Aderente]:

```
if (pagamento instanceof Pix) return BigDecimal.ZERO;
if (pagamento instanceof Cartao) return BigDecimal.valueOf(0.03);
throw new IllegalStateException("Tipo desconhecido");
```

[🔥 **Violação e Explicação**]: Não é extensível, é aberto a falhas em runtime ao adicionar novo tipo.

[✅ **Exemplo de Refatoração**]:

```
// Pattern matching / switch exaustivo moderno
return switch (pagamento) {
    case Pix p      -> BigDecimal.ZERO;
    case Cartao c   -> BigDecimal.valueOf(0.03);
};
```

Introduce Parameter Object

Trata *Data Clumps* onde o mesmo conjunto de parâmetros é copiado entre métodos.

[❌ **Código Não Aderente**]:

```
public void buscar(LocalDate inicio, LocalDate fim, String status, int pagina) { ... }
```

[🔥 **Violação e Explicação**]: Propensão a erros ao alterar assinatura; dados andam juntos mas não formam um objeto.

[✅ **Exemplo de Refatoração**]:

```
public record FiltroPedido(LocalDate inicio, LocalDate fim, String status) {}
public void buscar(FiltroPedido filtro, int pagina) { ... }
```

Quem aplica o quê

Situação	Quem	Skill
Aplicar refactoring em uma classe/método	sessão principal	esta skill
Revisar diff/PR com checklist de severidade	agent java-revisor	revisao-de-codigo-java
Remoção de parâmetro focada (passo-a-passo)	sessão principal	refactoring-remove-parameter
Limpar imports não usados	sessão principal	remover-imports-nao-usados
Centralizar configuração dispersa (Shotgun Surgery)	session/engenheiro-devops	java-architecture
Decidir onde mora um value object novo	session/java-construtor	arquitetura-limpa-java

```
with open("/mnt/data/SKILL_v1.2.0.md", "w", encoding="utf-8") as f: f.write(content)

print(f"File created successfully at {os.path.abspath('/mnt/data/SKILL_v1.2.0.md')}")
```

```
```python?code_reference&code_event_index=4
import os

content = """---
name: qualidade-codigo-java
description: "Application-side guide for clean code in Java – DRY, KISS, YAGNI, naming, immutability, `Optional`, st
license: MIT
metadata:
 author: https://github.com/srportto/srportto
 co-author: https://github.com/Jeffallan/claude-skills
 version: "1.2.0"
 domain: code-quality
 triggers: clean code, boas práticas, refatorar, DRY, KISS, YAGNI, imutabilidade, Optional, streams, Fowler, Object
 role: reference
 scope: code-quality
 output-format: code
 related-skills: revisao-de-codigo-java, padroes-de-projeto-java, refatorador-java, java-moderno

Qualidade de Código Java (clean code + refactoring)

Visão geral

Guia de **aplicação** de clean code em Java – DRY, KISS, YAGNI, nomenclatura, imutabilidade, `Optional`, streams, ex

Quando NÃO usar: para revisar um diff/PR com checklist por severidade, use `revisao-de-codigo-java` (ela referen

Clean code – princípios com exemplo
```

> **Coesão com `revisao-de-codigo-java`:** esta skill é o "lado ativo" (o **como** aplicar cada refactoring). A `rev`

> **Princípio-mestre (Clean Code for AI):** além de bom para humanos, todo código deste catálogo deve estar **otimiz**

### DRY — Don't Repeat Yourself

**[❌ Código Não Aderente]:**

```
```java
// logica de validacao duplicada em dois metodos
public void criarUsuario(UsuarioRequest req) {
    if (req.getEmail() == null || !req.getEmail().contains("@")) {
        throw new ValidationException("Email invalido");
    }
}

public void atualizarUsuario(UsuarioRequest req) {
    if (req.getEmail() == null || !req.getEmail().contains("@")) {
        throw new ValidationException("Email invalido");
    }
}
```

[🔥 Violação e Explicação]: mesma validação em 2 lugares — a 3ª ocorrência (em `importarEmLote`, por exemplo) confirma o padrão. Manter a duplicação significa N lugares para corrigir quando a regra mudar.

[✅ Exemplo de Refatoração]:

```
// fonte unica: metodo privado resolve sem criar interface/factory para o futuro
public class UsuarioService {
    public void criarUsuario(UsuarioRequest req) { validarEmail(req.getEmail()); /* ... */ }
    public void atualizarUsuario(UsuarioRequest req) { validarEmail(req.getEmail()); /* ... */ }

    private void validarEmail(String email) {
        if (email == null || !email.contains("@")) {
            throw new ValidationException("Email invalido");
        }
    }
}
```

KISS — Keep It Simple / YAGNI — You Aren't Gonna Need It

[❌ Código Não Aderente]:

```
// sobre-engenharia para 1 implementacao, sem segunda variacao a vista
public interface UserFactory {
    User createUser();
}

public class ConcreteUserFactory implements UserFactory {
    public User createUser() { return new User(); }
}
```

[🔥 Violação e Explicação]: interface + implementação única **"para o futuro"** é a abstração especulativa clássica (YAGNI). O custo (mais arquivos para ler, mais para o agente raciocinar) não traz benefício enquanto houver 1 variante.

[✅ Exemplo de Refatoração]:

```
// chamada direta; implemente a abstracao quando a segunda variacao aparecer de fato
public User createUser() { return new User(); }
```

Convenções de nomenclatura (Object Calisthenics: Don't Abbreviate)

A regra do Object Calisthenics orienta a nunca usar nomes abreviados, pois nomes com significado ajudam no entendimento e previnem falhas de design.

```
// Classes/Records: PascalCase
public class MarketService {}
public record Money(BigDecimal amount, Currency currency) {}

// Metodos/campos: camelCase
private final MarketRepository marketRepository;
public Market findBySlug(String slug) {}

// Constantes: UPPER_SNAKE_CASE
private static final int MAX_PAGE_SIZE = 100;
```

Nomes que revelam intenção e são grepáveis:

[❌ Código Não Aderente]:

```
// abreviaco es obscuras e nomes genericos nao sao grepaveis
public List<Produto> get(String s) { ... }
```

```
public boolean chk(String str) { ... }
private static final int N = 100;
public class Handler { public void handle(String p, String rng) { ... } }
```

[🚩 Violação e Explicação]: nomes genéricos (Handler, get, N, chk, p, rng) poluem a busca lexical, escondem a intenção e violam a regra "Don't Abbreviate". O agente tem que ler o corpo para descobrir o que o método faz. Se pesquisar pelo nome retorna coisas irrelevantes, o nome está ruim para a IA.

[✅ Exemplo de Refatoração]:

```
// nome diz o que faz; busca lexical (rg "AutorizacaoExpiradaHandler") cai direto
public List<Produto> buscarAtivosPorCategoria(String categoria) { ... }
public boolean precoEhValido(BigDecimal preco) { ... }
private static final int TAMANHO_MAXIMO_PAGINA = 100;
public class AutorizacaoExpiradaHandler {
    public void expirarAutorizacao(AutorizacaoId id, MotivoExpiracao motivo) { ... }
}
```

Nomes genéricos proibidos: Handler, Manager, Helper, Util, Data, Process, Info, Common, Base. Use nomes de domínio.

Imutabilidade

[❌ Código Não Aderente]:

```
// classe com setters publicos: estado mutavel depois da construcao
public class Market {
    private Long id;
    private String name;
    public void setId(Long id) { this.id = id; }
    public void setName(String name) { this.name = name; }
}
```

[🚩 Violação e Explicação]: setters públicos expõem o estado interno mutável; o chamador pode alterar id/name após a construção, quebrando invariantes do domínio e criando bugs sutis em fluxos assíncronos.

[✅ Exemplo de Refatoração]:

```
// record para DTOs e value objects (imutavel, equals/hashCode/toString gerados)
public record MarketDto(Long id, String name, MarketStatus status) {}

// ou classe com final fields e getters only
public class Market {
    private final Long id;
    private final String name;
    // getters only, no setters
}
```

Optional — uso correto

[❌ Código Não Aderente]:

```
public Market buscarPorSlug(String slug) {
    Optional<Market> market = marketRepository.findBySlug(slug);
    return market.get(); // NoSuchElementException se vazio
}
```

[🚩 Violação e Explicação]: Optional.get() sem .orElse().orElseThrow().isPresent() joga a decisão para erro em runtime sem chance do chamador reagir.

[✅ Exemplo de Refatoração]:

```
public Market buscarPorSlug(String slug) {
    return marketRepository.findBySlug(slug)
        .orElseThrow(() -> new EntityNotFoundException("Market not found: " + slug));
}
```

Streams — pipelines curtos, sem efeito colateral

[❌ Código Não Aderente]:

```
List<String> nomesAtivos = new ArrayList<>();
markets.stream().forEach(m -> {
    if (m.isAtivo()) {
        nomesAtivos.add(m.name().toUpperCase());
    }
});
```

[🚩 Violação e Explicação]: forEach capturando variável externa é anti-pattern clássico; força rastrear o efeito colateral e impede paralelização.

[✓ Exemplo de Refatoração]:

```
List<String> names = markets.stream()
    .filter(Market::isAtivo)
    .map(m -> m.name().toUpperCase())
    .toList();
```

Exception handling

- Use **unchecked exceptions** para erros de domínio (`BusinessException`).
- **Crie exceções específicas** em vez de `RuntimeException` genérica.
- **Evite** `catch (Exception ex) amplo`.
- **Sempre preserve a causa** (`throw new ApplicationException(msg, e)`).

[✗ Código Não Aderente]:

```
try {
    integracaoClient.enviar(pedido);
} catch (IOException e) {
    throw new RuntimeException(e.getMessage()); // Perde a causa (e)
}
```

[🔥 Violação e Explicação]: perde a `Throwable` `cause` (stack trace) e a mensagem original. Mensagem vaga força a IA a gastar turnos extras para descobrir o que deu errado.

[✓ Exemplo de Refatoração]:

```
try {
    integracaoClient.enviar(pedido);
} catch (IOException e) {
    throw new ApplicationException("Falha ao enviar pedido " + pedido.id(), e);
}
```

Genéricos e type safety (Tipos explícitos para IA)**[✗ Código Não Aderente]:**

```
public Map indexById(Collection items) { ... } // sem type safety
```

[🔥 Violação e Explicação]: código sem anotações de tipo ou com raw types obriga agentes e humanos a inferirem o que entra e sai, gerando falhas. O agente poupa trabalho de descoberta em códigos tipados.

[✓ Exemplo de Refatoração]:

```
public <T extends Identifiable> Map<Long, T> indexById(Collection<T> items) { ... }
```

Tell, Don't Ask & No Getters/Setters (Object Calisthenics)

O código cliente não deve perguntar o estado interno de um objeto para tomar decisões. A regra "No Getters/Setters/Properties" foca no encapsulamento de comportamentos e evita a exposição indevida que viola a orientação a objetos. Uma classe exposta à manipulação de estado por fora gera domínio anêmico e separa os dados do comportamento.

[✗ Código Não Aderente]:

```
// Dominio anemico: o objeto e um saco de dados, a acao e feita de fora
public void aplicarDesconto(Produto produto) {
    if (produto.getPreco() > 50) {
        produto.setPreco(produto.getPreco() - 10);
    }
}
```

[🔥 Violação e Explicação]:

1. Ferimento de encapsulamento e do princípio "Tell, Don't Ask". O cliente pergunta o preço para decidir a regra.
2. O agente LLM precisa ler 2 arquivos (`Produto` + `Service`) para entender a regra.
3. Se a regra mudar, será necessário caçar onde esse getter/setter foi usado no código.

[✓ Exemplo de Refatoração]:

```
public class Produto {
    private BigDecimal preco;

    // O objeto controla e protege sua propria regra
    public void aplicarDesconto(BigDecimal valorDesconto) {
        if (this.preco.compareTo(new BigDecimal("50")) > 0) {
```

```

        this.preco = this.preco.subtract(valorDesconto);
    }
}

// O cliente so envia o comando
public void processar(Produto produto) {
    produto.aplicarDesconto(new BigDecimal("10"));
}

```

Primitive Obsession & Wrap All Primitives (Object Calisthenics)

Não use tipos primitivos para representar conceitos com comportamento. A regra "Wrap All Primitives And Strings" determina que variáveis primitivas com comportamento específico de domínio (como validações próprias) devem ser encapsuladas em Objetos.

[❌ Código Não Aderente]:

```

public class Pessoa {
    private String cpf; // Obsessao por tipo primitivo

    public Pessoa(String cpf) {
        if (cpf == null || cpf.length() != 11) throw new IllegalArgumentException();
        this.cpf = cpf;
    }
}

```

[🔥 **Violação e Explicação**]: O CPF em formato String espalha lógica de validação. O agente de IA precisa inferir o formato, consumindo janela de contexto. A regra de validação não pertence estruturalmente à Pessoa.

[✅ Exemplo de Refatoração]:

```

public record Cpf(String numero) { // Value object imutavel que contem suas regras
    public Cpf {
        if (numero == null || numero.length() != 11) {
            throw new IllegalArgumentException("CPF Invalido");
        }
    }
}

public class Pessoa {
    private Cpf cpf; // Propriedade tipada e protegida
}

```

First Class Collections (Object Calisthenics)

Qualquer classe que contenha uma coleção não deve conter outras variáveis de membro. Se você tem um conjunto de elementos, crie uma classe dedicada exclusivamente a essa coleção.

[❌ Código Não Aderente]:

```

public class Empresa {
    private String razaoSocial;
    private List<Funcionario> funcionarios; // Colecao misturada com outros atributos

    // Metodos que manipulam a lista se misturam com metodos da empresa
    public List<Funcionario> buscarContadores() {
        return funcionarios.stream().filter(f -> f.getCargo().equals("contador")).toList();
    }
}

```

[🔥 **Violação e Explicação**]: Os comportamentos de filtro e agrupamento de funcionários poluem a classe Empresa.

[✅ Exemplo de Refatoração]:

```

public class QuadroFuncionarios { // First Class Collection
    private final List<Funcionario> funcionarios;

    public QuadroFuncionarios(List<Funcionario> funcionarios) {
        this.funcionarios = funcionarios;
    }

    // Comportamentos especificos tem um lar
    public List<Funcionario> buscarContadores() {
        return funcionarios.stream().filter(f -> f.getCargo().equals("contador")).toList();
    }
}

public class Empresa {
    private String razaoSocial;
}

```



```
private QuadroFuncionarios quadro;  
}
```

One Dot Per Line / Law of Demeter (Object Calisthenics)

Evite cadeias extensas de chamadas que atravessam vários objetos. Se você usa mais de um ponto na mesma linha, o seu objeto é um intermediário sabendo demais sobre a estrutura dos outros.

[❌ Código Não Aderente]:

```
// Navegando a estrutura interna (quebra de encapsulamento)  
String nomeChefe = funcionario.getDepartamento().getChefe().getNome();
```

[🔥 Violação e Explicação]: A estrutura interna do objeto fica exposta, dificultando a leitura e acoplando fortemente as classes.

[✅ Exemplo de Refatoração]:

```
// O objeto expressa intencoes via metodos comportamentais diretos  
String nomeChefe = funcionario.getNomeChefeDepartamento();
```

No Classes With More Than Two Instance Variables (Alta Coesão)

Classes não devem ter mais do que duas variáveis de instância, forçando um alto nível de coesão, composição e abstração. O objetivo prático é agrupar variáveis em lógicas menores quando a classe assume muitas responsabilidades.

[❌ Código Não Aderente]:

```
public class Funcionario {  
    private String nome;  
    private int idade;  
    private String cargo;  
    private String departamento;  
}
```

[🔥 Violação e Explicação]: A classe tem muitos atributos e assume informações tanto pessoais quanto contratuais.

[✅ Exemplo de Refatoração]:

```
public class Funcionario {  
    private InformacoesPessoais dadosPessoais; // Agrupa nome e idade  
    private InformacoesTrabalho dadosTrabalho; // Agrupa cargo e departamento  
}
```

Replace Magic Number with Symbolic Constant

Qualquer literal numérico ou `String` com **significado de domínio** é proibido no meio de validações. Deve virar constante nomeada.

[❌ Código Não Aderente]:

```
if (valor.compareTo(new BigDecimal("50000")) > 0) { ... }
```

[🔥 Violação e Explicação]: Magic Numbers escondem a regra. O agente precisa adivinhar o motivo da limitação.

[✅ Exemplo de Refatoração]:

```
// Constante documenta a regra  
private static final BigDecimal LIMITE_MAXIMO_PIX = new BigDecimal("50000");  
if (valor.compareTo(LIMITE_MAXIMO_PIX) > 0) { ... }
```

Guard Clauses & Don't Use "Else" (Object Calisthenics)

O uso de `else` é **proibido**. Assuma o fluxo padrão e faça validações através de Fail-Fast, Early Return ou Guard Clauses. Cada nível de indentação extra multiplica o custo cognitivo.

[❌ Código Não Aderente]:

```
public void processar(Pedido pedido) {  
    if (pedido != null) {  
        if (!pedido.getItems().isEmpty()) {  
            executar(pedido);  
        } else {  
            throw new BusinessException("sem itens");  
        }  
    }  
}
```

[🔥 Violação e Explicação]: Aninhamento de ifs aumenta a complexidade e polui a legibilidade. Para LLMs, o "else" desvia a atenção da lógica contínua.

[✓ Exemplo de Refatoração]:

```
public void processar(Pedido pedido) {
    if (pedido == null) return;

    if (pedido.getItems().isEmpty()) {
        throw new BusinessException("sem itens");
    }

    executar(pedido);
}
```

Clean Code for AI (Arquitetura para o Agente)

Regras vitais para quando o LLM interage e edita a base de código:

1. **Only One Level Of Indentation Per Method & Manter Métodos Pequenos:** Métodos curtos, contendo de 4 a 20 linhas ou 1 nível de indentação, cabem na mesma tool call do LLM, evitando perda de contexto.
2. **SRP (Responsabilidade Única):** Permite edições isoladas via AI sem efeito colateral destrutivo.
3. **Comentários de Proveniência:** Embora comentários óbvios (ex: `// incrementa i`) consumam tokens de IA à toa e devam sumir, documentações que explicam o *motivo* da decisão de negócio são cruciais.
4. **Testes que o agente consegue rodar:** Testes automatizados rápidos e headless (TDD) atuam como bússola, diferenciando o agente ágil do que trabalha "chutando".
5. **Injeção de Dependências:** Instanciar dependências hardcoded impede o isolamento no momento do teste. Usar DI facilita a refatoração e as suítes de teste.

Refactorings do Fowler e Bloaters — guia rápido

Além dos princípios, usamos técnicas do catálogo Refactoring Guru para sanar "Smells".

Remove Parameter**[✗ Código Não Aderente]:**

```
// "isCloud" e recebido mas nunca influencia o resultado
public Backend selecionarBackend(long tableId, ConnectContext context, boolean isCloud) { ... }
```

[🔥 Violação e Explicação]: Parâmetro morto polui a assinatura e é uma abstração especulativa.

[✓ Exemplo de Refatoração]:

```
public Backend selecionarBackend(long tableId, ConnectContext context) { ... }
```

Extract Method

Resolvendo o problema do *Long Method* (Bloater).

[✗ Código Não Aderente]:

```
public void processar(Pedido pedido) {
    // 20 linhas de validacao aqui
    // logica principal
}
```

[🔥 Violação e Explicação]: Validações inline espessas dificultam ler o fluxo real de domínio. O agente perde o foco de qual ação principal é realizada.

[✓ Exemplo de Refatoração]:

```
public void processar(Pedido pedido) {
    validarPedido(pedido);
    // logica principal
}
```

Replace Conditional with Polymorphism

Resolvendo o cheiro de *Switch Statements* ou longos *ifs* encadeados que checam tipos.

[✗ Código Não Aderente]:

```
if (pagamento instanceof Pix) return BigDecimal.ZERO;
if (pagamento instanceof Cartao) return BigDecimal.valueOf(0.03);
throw new IllegalStateException("Tipo desconhecido");
```

[🔥 **Violação e Explicação**]: Não é extensível, é aberto a falhas em runtime ao adicionar novo tipo.

[✅ **Exemplo de Refatoração**]:

```
// Pattern matching / switch exaustivo moderno
return switch (pagamento) {
    case Pix p      -> BigDecimal.ZERO;
    case Cartao c   -> BigDecimal.valueOf(0.03);
};
```

Introduce Parameter Object

Trata *Data Clumps* onde o mesmo conjunto de parâmetros é copiado entre métodos.

[❌ **Código Não Aderente**]:

```
public void buscar(LocalDate inicio, LocalDate fim, String status, int pagina) { ... }
```

[🔥 **Violação e Explicação**]: Propensão a erros ao alterar assinatura; dados andam juntos mas não formam um objeto.

[✅ **Exemplo de Refatoração**]:

```
public record FiltroPedido(LocalDate inicio, LocalDate fim, String status) {}
public void buscar(FiltroPedido filtro, int pagina) { ... }
```

Quem aplica o quê

Situação	Quem	Skill
Aplicar refactoring em uma classe/método	sessão principal	esta skill
Revisar diff/PR com checklist de severidade	agent java-revisor	revisao-de-codigo-java
Remoção de parâmetro focada (passo-a-passo)	sessão principal	refactoring-remove-parameter
Limpar imports não usados	sessão principal	remover-imports-nao-usados
Centralizar configuração dispersa (Shotgun Surgery)	session/engenheiro-devops	java-architecture
Decidir onde mora um value object novo	session/java-construtor	arquitetura-limpa-java

```
with open("/mnt/data/SKILL_v1.2.0.md", "w", encoding="utf-8") as f: f.write(content)

print(f"File created successfully at {os.path.abspath('/mnt/data/SKILL_v1.2.0.md')}")
```

```
```python?code_reference&code_event_index=5
import os

content = """---
name: qualidade-codigo-java
description: "Application-side guide for clean code in Java – DRY, KISS, YAGNI, naming, immutability, `Optional`, st
license: MIT
metadata:
 author: https://github.com/srportto/srportto
 co-author: https://github.com/Jeffallan/claude-skills
 version: "1.2.0"
 domain: code-quality
 triggers: clean code, boas práticas, refatorar, DRY, KISS, YAGNI, imutabilidade, Optional, streams, Fowler, Object
 role: reference
 scope: code-quality
 output-format: code
 related-skills: revisao-de-codigo-java, padroes-de-projeto-java, refatorador-java, java-moderno

Qualidade de Código Java (clean code + refactoring)

Visão geral

Guia de **aplicação** de clean code em Java – DRY, KISS, YAGNI, nomenclatura, imutabilidade, `Optional`, streams, ex

Quando NÃO usar: para revisar um diff/PR com checklist por severidade, use `revisao-de-codigo-java` (ela referen

Clean code – princípios com exemplo
```

> **Coesão com `revisao-de-codigo-java`:** esta skill é o "lado ativo" (o **como** aplicar cada refactoring). A `rev`

> **Princípio-mestre (Clean Code for AI):** além de bom para humanos, todo código deste catálogo deve estar **otimiz**

### DRY — Don't Repeat Yourself

**[❌ Código Não Aderente]:**

```
```java
// logica de validacao duplicada em dois metodos
public void criarUsuario(UsuarioRequest req) {
    if (req.getEmail() == null || !req.getEmail().contains("@")) {
        throw new ValidationException("Email invalido");
    }
}

public void atualizarUsuario(UsuarioRequest req) {
    if (req.getEmail() == null || !req.getEmail().contains("@")) {
        throw new ValidationException("Email invalido");
    }
}
```

[🔥 Violação e Explicação]: mesma validação em 2 lugares — a 3ª ocorrência (em `importarEmLote`, por exemplo) confirma o padrão. Manter a duplicação significa N lugares para corrigir quando a regra mudar.

[✅ Exemplo de Refatoração]:

```
// fonte unica: metodo privado resolve sem criar interface/factory para o futuro
public class UsuarioService {
    public void criarUsuario(UsuarioRequest req) { validarEmail(req.getEmail()); /* ... */ }
    public void atualizarUsuario(UsuarioRequest req) { validarEmail(req.getEmail()); /* ... */ }

    private void validarEmail(String email) {
        if (email == null || !email.contains("@")) {
            throw new ValidationException("Email invalido");
        }
    }
}
```

KISS — Keep It Simple / YAGNI — You Aren't Gonna Need It

[❌ Código Não Aderente]:

```
// sobre-engenharia para 1 implementacao, sem segunda variacao a vista
public interface UserFactory {
    User createUser();
}

public class ConcreteUserFactory implements UserFactory {
    public User createUser() { return new User(); }
}
```

[🔥 Violação e Explicação]: interface + implementação única **"para o futuro"** é a abstração especulativa clássica (YAGNI). O custo (mais arquivos para ler, mais para o agente raciocinar) não traz benefício enquanto houver 1 variante.

[✅ Exemplo de Refatoração]:

```
// chamada direta; implemente a abstracao quando a segunda variacao aparecer de fato
public User createUser() { return new User(); }
```

Convenções de nomenclatura (Object Calisthenics: Don't Abbreviate)

A regra do Object Calisthenics orienta a nunca usar nomes abreviados, pois nomes com significado ajudam no entendimento e previnem falhas de design.

```
// Classes/Records: PascalCase
public class MarketService {}
public record Money(BigDecimal amount, Currency currency) {}

// Metodos/campos: camelCase
private final MarketRepository marketRepository;
public Market findBySlug(String slug) {}

// Constantes: UPPER_SNAKE_CASE
private static final int MAX_PAGE_SIZE = 100;
```

Nomes que revelam intenção e são grepáveis:

[❌ Código Não Aderente]:

```
// abreviaco es obscuras e nomes genericos nao sao grepaveis
public List<Produto> get(String s) { ... }
```

```
public boolean chk(String str) { ... }
private static final int N = 100;
public class Handler { public void handle(String p, String rng) { ... } }
```

[🚩 Violação e Explicação]: nomes genéricos (Handler, get, N, chk, p, rng) poluem a busca lexical, escondem a intenção e violam a regra "Don't Abbreviate". O agente tem que ler o corpo para descobrir o que o método faz. Se pesquisar pelo nome retorna coisas irrelevantes, o nome está ruim para a IA.

[✅ Exemplo de Refatoração]:

```
// nome diz o que faz; busca lexical (rg "AutorizacaoExpiradaHandler") cai direto
public List<Produto> buscarAtivosPorCategoria(String categoria) { ... }
public boolean precoEhValido(BigDecimal preco) { ... }
private static final int TAMANHO_MAXIMO_PAGINA = 100;
public class AutorizacaoExpiradaHandler {
    public void expirarAutorizacao(AutorizacaoId id, MotivoExpiracao motivo) { ... }
}
```

Nomes genéricos proibidos: Handler, Manager, Helper, Util, Data, Process, Info, Common, Base. Use nomes de domínio.

Imutabilidade

[❌ Código Não Aderente]:

```
// classe com setters publicos: estado mutavel depois da construcao
public class Market {
    private Long id;
    private String name;
    public void setId(Long id) { this.id = id; }
    public void setName(String name) { this.name = name; }
}
```

[🚩 Violação e Explicação]: setters públicos expõem o estado interno mutável; o chamador pode alterar id/name após a construção, quebrando invariantes do domínio e criando bugs sutis em fluxos assíncronos.

[✅ Exemplo de Refatoração]:

```
// record para DTOs e value objects (imutavel, equals/hashCode/toString gerados)
public record MarketDto(Long id, String name, MarketStatus status) {}

// ou classe com final fields e getters only
public class Market {
    private final Long id;
    private final String name;
    // getters only, no setters
}
```

Optional — uso correto

[❌ Código Não Aderente]:

```
public Market buscarPorSlug(String slug) {
    Optional<Market> market = marketRepository.findBySlug(slug);
    return market.get(); // NoSuchElementException se vazio
}
```

[🚩 Violação e Explicação]: Optional.get() sem .orElse().orElseThrow().isPresent() joga a decisão para erro em runtime sem chance do chamador reagir.

[✅ Exemplo de Refatoração]:

```
public Market buscarPorSlug(String slug) {
    return marketRepository.findBySlug(slug)
        .orElseThrow(() -> new EntityNotFoundException("Market not found: " + slug));
}
```

Streams — pipelines curtos, sem efeito colateral

[❌ Código Não Aderente]:

```
List<String> nomesAtivos = new ArrayList<>();
markets.stream().forEach(m -> {
    if (m.isAtivo()) {
        nomesAtivos.add(m.name().toUpperCase());
    }
});
```

[🚩 Violação e Explicação]: forEach capturando variável externa é anti-pattern clássico; força rastrear o efeito colateral e impede paralelização.

[✓ Exemplo de Refatoração]:

```
List<String> names = markets.stream()
    .filter(Market::isAtivo)
    .map(m -> m.name().toUpperCase())
    .toList();
```

Exception handling

- Use **unchecked exceptions** para erros de domínio (`BusinessException`).
- **Crie exceções específicas** em vez de `RuntimeException` genérica.
- **Evite** `catch (Exception ex) amplo`.
- **Sempre preserve a causa** (`throw new ApplicationException(msg, e)`).

[✗ Código Não Aderente]:

```
try {
    integracaoClient.enviar(pedido);
} catch (IOException e) {
    throw new RuntimeException(e.getMessage()); // Perde a causa (e)
}
```

[🔥 Violação e Explicação]: perde a `Throwable` `cause` (stack trace) e a mensagem original. Mensagem vaga força a IA a gastar turnos extras para descobrir o que deu errado.

[✓ Exemplo de Refatoração]:

```
try {
    integracaoClient.enviar(pedido);
} catch (IOException e) {
    throw new ApplicationException("Falha ao enviar pedido " + pedido.id(), e);
}
```

Genéricos e type safety (Tipos explícitos para IA)**[✗ Código Não Aderente]:**

```
public Map indexById(Collection items) { ... } // sem type safety
```

[🔥 Violação e Explicação]: código sem anotações de tipo ou com raw types obriga agentes e humanos a inferirem o que entra e sai, gerando falhas. O agente poupa trabalho de descoberta em códigos tipados.

[✓ Exemplo de Refatoração]:

```
public <T extends Identifiable> Map<Long, T> indexById(Collection<T> items) { ... }
```

Tell, Don't Ask & No Getters/Setters (Object Calisthenics)

O código cliente não deve perguntar o estado interno de um objeto para tomar decisões. A regra "No Getters/Setters/Properties" foca no encapsulamento de comportamentos e evita a exposição indevida que viola a orientação a objetos. Uma classe exposta à manipulação de estado por fora gera domínio anêmico e separa os dados do comportamento.

[✗ Código Não Aderente]:

```
// Dominio anemico: o objeto e um saco de dados, a acao e feita de fora
public void aplicarDesconto(Produto produto) {
    if (produto.getPreco() > 50) {
        produto.setPreco(produto.getPreco() - 10);
    }
}
```

[🔥 Violação e Explicação]:

1. Ferimento de encapsulamento e do princípio "Tell, Don't Ask". O cliente pergunta o preço para decidir a regra.
2. O agente LLM precisa ler 2 arquivos (`Produto` + `Service`) para entender a regra.
3. Se a regra mudar, será necessário caçar onde esse getter/setter foi usado no código.

[✓ Exemplo de Refatoração]:

```
public class Produto {
    private BigDecimal preco;

    // O objeto controla e protege sua propria regra
    public void aplicarDesconto(BigDecimal valorDesconto) {
        if (this.preco.compareTo(new BigDecimal("50")) > 0) {
```

```

        this.preco = this.preco.subtract(valorDesconto);
    }
}

// O cliente so envia o comando
public void processar(Produto produto) {
    produto.aplicarDesconto(new BigDecimal("10"));
}

```

Primitive Obsession & Wrap All Primitives (Object Calisthenics)

Não use tipos primitivos para representar conceitos com comportamento. A regra "Wrap All Primitives And Strings" determina que variáveis primitivas com comportamento específico de domínio (como validações próprias) devem ser encapsuladas em Objetos.

[❌ Código Não Aderente]:

```

public class Pessoa {
    private String cpf; // Obsessao por tipo primitivo

    public Pessoa(String cpf) {
        if (cpf == null || cpf.length() != 11) throw new IllegalArgumentException();
        this.cpf = cpf;
    }
}

```

[🔥 **Violação e Explicação**]: O CPF em formato String espalha lógica de validação. O agente de IA precisa inferir o formato, consumindo janela de contexto. A regra de validação não pertence estruturalmente à Pessoa.

[✅ Exemplo de Refatoração]:

```

public record Cpf(String numero) { // Value object imutavel que contem suas regras
    public Cpf {
        if (numero == null || numero.length() != 11) {
            throw new IllegalArgumentException("CPF Invalido");
        }
    }
}

public class Pessoa {
    private Cpf cpf; // Propriedade tipada e protegida
}

```

First Class Collections (Object Calisthenics)

Qualquer classe que contenha uma coleção não deve conter outras variáveis de membro. Se você tem um conjunto de elementos, crie uma classe dedicada exclusivamente a essa coleção.

[❌ Código Não Aderente]:

```

public class Empresa {
    private String razaoSocial;
    private List<Funcionario> funcionarios; // Colecao misturada com outros atributos

    // Metodos que manipulam a lista se misturam com metodos da empresa
    public List<Funcionario> buscarContadores() {
        return funcionarios.stream().filter(f -> f.getCargo().equals("contador")).toList();
    }
}

```

[🔥 **Violação e Explicação**]: Os comportamentos de filtro e agrupamento de funcionários poluem a classe Empresa.

[✅ Exemplo de Refatoração]:

```

public class QuadroFuncionarios { // First Class Collection
    private final List<Funcionario> funcionarios;

    public QuadroFuncionarios(List<Funcionario> funcionarios) {
        this.funcionarios = funcionarios;
    }

    // Comportamentos especificos tem um lar
    public List<Funcionario> buscarContadores() {
        return funcionarios.stream().filter(f -> f.getCargo().equals("contador")).toList();
    }
}

public class Empresa {
    private String razaoSocial;
}

```

```
private QuadroFuncionarios quadro;
}
```

One Dot Per Line / Law of Demeter (Object Calisthenics)

Evite cadeias extensas de chamadas que atravessam vários objetos. Se você usa mais de um ponto na mesma linha, o seu objeto é um intermediário sabendo demais sobre a estrutura dos outros.

[❌ Código Não Aderente]:

```
// Navegando a estrutura interna (quebra de encapsulamento)
String nomeChefe = funcionario.getDepartamento().getChefe().getNome();
```

[🔥 Violação e Explicação]: A estrutura interna do objeto fica exposta, dificultando a leitura e acoplando fortemente as classes.

[✅ Exemplo de Refatoração]:

```
// O objeto expressa intencoes via metodos comportamentais diretos
String nomeChefe = funcionario.getNomeChefeDepartamento();
```

No Classes With More Than Two Instance Variables (Alta Coesão)

Classes não devem ter mais do que duas variáveis de instância, forçando um alto nível de coesão, composição e abstração. O objetivo prático é agrupar variáveis em lógicas menores quando a classe assume muitas responsabilidades.

[❌ Código Não Aderente]:

```
public class Funcionario {
    private String nome;
    private int idade;
    private String cargo;
    private String departamento;
}
```

[🔥 Violação e Explicação]: A classe tem muitos atributos e assume informações tanto pessoais quanto contratuais.

[✅ Exemplo de Refatoração]:

```
public class Funcionario {
    private InformacoesPessoais dadosPessoais; // Agrupa nome e idade
    private InformacoesTrabalho dadosTrabalho; // Agrupa cargo e departamento
}
```

Replace Magic Number with Symbolic Constant

Qualquer literal numérico ou `String` com **significado de domínio** é proibido no meio de validações. Deve virar constante nomeada.

[❌ Código Não Aderente]:

```
if (valor.compareTo(new BigDecimal("50000")) > 0) { ... }
```

[🔥 Violação e Explicação]: Magic Numbers escondem a regra. O agente precisa adivinhar o motivo da limitação.

[✅ Exemplo de Refatoração]:

```
// Constante documenta a regra
private static final BigDecimal LIMITE_MAXIMO_PIX = new BigDecimal("50000");
if (valor.compareTo(LIMITE_MAXIMO_PIX) > 0) { ... }
```

Guard Clauses & Don't Use "Else" (Object Calisthenics)

O uso de `else` é **proibido**. Assuma o fluxo padrão e faça validações através de Fail-Fast, Early Return ou Guard Clauses. Cada nível de indentação extra multiplica o custo cognitivo.

[❌ Código Não Aderente]:

```
public void processar(Pedido pedido) {
    if (pedido != null) {
        if (!pedido.getItems().isEmpty()) {
            executar(pedido);
        } else {
            throw new BusinessException("sem itens");
        }
    }
}
```

[🔥 Violação e Explicação]: Aninhamento de ifs aumenta a complexidade e polui a legibilidade. Para LLMs, o "else" desvia a atenção da lógica contínua.

[✓ Exemplo de Refatoração]:

```
public void processar(Pedido pedido) {
    if (pedido == null) return;

    if (pedido.getItems().isEmpty()) {
        throw new BusinessException("sem itens");
    }

    executar(pedido);
}
```

Clean Code for AI (Arquitetura para o Agente)

Regras vitais para quando o LLM interage e edita a base de código:

1. **Only One Level Of Indentation Per Method & Manter Métodos Pequenos:** Métodos curtos, contendo de 4 a 20 linhas ou 1 nível de indentação, cabem na mesma tool call do LLM, evitando perda de contexto.
2. **SRP (Responsabilidade Única):** Permite edições isoladas via AI sem efeito colateral destrutivo.
3. **Comentários de Proveniência:** Embora comentários óbvios (ex: `// incrementa i`) consumam tokens de IA à toa e devam sumir, documentações que explicam o *motivo* da decisão de negócio são cruciais.
4. **Testes que o agente consegue rodar:** Testes automatizados rápidos e headless (TDD) atuam como bússola, diferenciando o agente ágil do que trabalha "chutando".
5. **Injeção de Dependências:** Instanciar dependências hardcoded impede o isolamento no momento do teste. Usar DI facilita a refatoração e as suítes de teste.

Refactorings do Fowler e Bloaters — guia rápido

Além dos princípios, usamos técnicas do catálogo Refactoring Guru para sanar "Smells".

Remove Parameter**[✗ Código Não Aderente]:**

```
// "isCloud" e recebido mas nunca influencia o resultado
public Backend selecionarBackend(long tableId, ConnectContext context, boolean isCloud) { ... }
```

[🔥 Violação e Explicação]: Parâmetro morto polui a assinatura e é uma abstração especulativa.

[✓ Exemplo de Refatoração]:

```
public Backend selecionarBackend(long tableId, ConnectContext context) { ... }
```

Extract Method

Resolvendo o problema do *Long Method* (Bloater).

[✗ Código Não Aderente]:

```
public void processar(Pedido pedido) {
    // 20 linhas de validacao aqui
    // logica principal
}
```

[🔥 Violação e Explicação]: Validações inline espessas dificultam ler o fluxo real de domínio. O agente perde o foco de qual ação principal é realizada.

[✓ Exemplo de Refatoração]:

```
public void processar(Pedido pedido) {
    validarPedido(pedido);
    // logica principal
}
```

Replace Conditional with Polymorphism

Resolvendo o cheiro de *Switch Statements* ou longos *ifs* encadeados que checam tipos.

[✗ Código Não Aderente]:

```
if (pagamento instanceof Pix) return BigDecimal.ZERO;
if (pagamento instanceof Cartao) return BigDecimal.valueOf(0.03);
throw new IllegalStateException("Tipo desconhecido");
```

[🔥 **Violação e Explicação**]: Não é extensível, é aberto a falhas em runtime ao adicionar novo tipo.

[✅ **Exemplo de Refatoração**]:

```
// Pattern matching / switch exaustivo moderno
return switch (pagamento) {
    case Pix p      -> BigDecimal.ZERO;
    case Cartao c   -> BigDecimal.valueOf(0.03);
};
```

Introduce Parameter Object

Trata *Data Clumps* onde o mesmo conjunto de parâmetros é copiado entre métodos.

[❌ **Código Não Aderente**]:

```
public void buscar(LocalDate inicio, LocalDate fim, String status, int pagina) { ... }
```

[🔥 **Violação e Explicação**]: Propensão a erros ao alterar assinatura; dados andam juntos mas não formam um objeto.

[✅ **Exemplo de Refatoração**]:

```
public record FiltroPedido(LocalDate inicio, LocalDate fim, String status) {}
public void buscar(FiltroPedido filtro, int pagina) { ... }
```

Quem aplica o quê

Situação	Quem	Skill
Aplicar refactoring em uma classe/método	sessão principal	esta skill
Revisar diff/PR com checklist de severidade	agent java-revisor	revisao-de-codigo-java
Remoção de parâmetro focada (passo-a-passo)	sessão principal	refactoring-remove-parameter
Limpar imports não usados	sessão principal	remover-imports-nao-usados
Centralizar configuração dispersa (Shotgun Surgery)	session/engenheiro-devops	java-architecture
Decidir onde mora um value object novo	session/java-construtor	arquitetura-limpa-java

```
with open("/tmp/SKILL_v1.2.0.md", "w", encoding="utf-8") as f: f.write(content)

print(f"File created successfully at {os.path.abspath('/tmp/SKILL_v1.2.0.md')}")
```

```
```python?code_reference&code_event_index=6
import os

content = """---
name: qualidade-codigo-java
description: "Application-side guide for clean code in Java – DRY, KISS, YAGNI, naming, immutability, `Optional`, st
license: MIT
metadata:
 author: https://github.com/srportto/srportto
 co-author: https://github.com/Jeffallan/claude-skills
 version: "1.2.0"
 domain: code-quality
 triggers: clean code, boas práticas, refatorar, DRY, KISS, YAGNI, imutabilidade, Optional, streams, Fowler, Object
 role: reference
 scope: code-quality
 output-format: code
 related-skills: revisao-de-codigo-java, padroes-de-projeto-java, refatorador-java, java-moderno

Qualidade de Código Java (clean code + refactoring)

Visão geral

Guia de **aplicação** de clean code em Java – DRY, KISS, YAGNI, nomenclatura, imutabilidade, `Optional`, streams, ex

Quando NÃO usar: para revisar um diff/PR com checklist por severidade, use `revisao-de-codigo-java` (ela referen

Clean code – princípios com exemplo
```

> **Coesão com `revisao-de-codigo-java`:** esta skill é o "lado ativo" (o **como** aplicar cada refactoring). A `rev`

> **Princípio-mestre (Clean Code for AI):** além de bom para humanos, todo código deste catálogo deve estar **otimiz**

### DRY — Don't Repeat Yourself

**[❌ Código Não Aderente]:**

```
```java
// logica de validacao duplicada em dois metodos
public void criarUsuario(UsuarioRequest req) {
    if (req.getEmail() == null || !req.getEmail().contains("@")) {
        throw new ValidationException("Email invalido");
    }
}

public void atualizarUsuario(UsuarioRequest req) {
    if (req.getEmail() == null || !req.getEmail().contains("@")) {
        throw new ValidationException("Email invalido");
    }
}
```

[🔥 Violação e Explicação]: mesma validação em 2 lugares — a 3ª ocorrência (em `importarEmLote`, por exemplo) confirma o padrão. Manter a duplicação significa N lugares para corrigir quando a regra mudar.

[✅ Exemplo de Refatoração]:

```
// fonte unica: metodo privado resolve sem criar interface/factory para o futuro
public class UsuarioService {
    public void criarUsuario(UsuarioRequest req) { validarEmail(req.getEmail()); /* ... */ }
    public void atualizarUsuario(UsuarioRequest req) { validarEmail(req.getEmail()); /* ... */ }

    private void validarEmail(String email) {
        if (email == null || !email.contains("@")) {
            throw new ValidationException("Email invalido");
        }
    }
}
```

KISS — Keep It Simple / YAGNI — You Aren't Gonna Need It

[❌ Código Não Aderente]:

```
// sobre-engenharia para 1 implementacao, sem segunda variacao a vista
public interface UserFactory {
    User createUser();
}

public class ConcreteUserFactory implements UserFactory {
    public User createUser() { return new User(); }
}
```

[🔥 Violação e Explicação]: interface + implementação única **"para o futuro"** é a abstração especulativa clássica (YAGNI). O custo (mais arquivos para ler, mais para o agente raciocinar) não traz benefício enquanto houver 1 variante.

[✅ Exemplo de Refatoração]:

```
// chamada direta; implemente a abstracao quando a segunda variacao aparecer de fato
public User createUser() { return new User(); }
```

Convenções de nomenclatura (Object Calisthenics: Don't Abbreviate)

A regra do Object Calisthenics orienta a nunca usar nomes abreviados, pois nomes com significado ajudam no entendimento e previnem falhas de design.

```
// Classes/Records: PascalCase
public class MarketService {}
public record Money(BigDecimal amount, Currency currency) {}

// Metodos/campos: camelCase
private final MarketRepository marketRepository;
public Market findBySlug(String slug) {}

// Constantes: UPPER_SNAKE_CASE
private static final int MAX_PAGE_SIZE = 100;
```

Nomes que revelam intenção e são grepáveis:

[❌ Código Não Aderente]:

```
// abreviaco es obscuras e nomes genericos nao sao grepaveis
public List<Produto> get(String s) { ... }
```

```
public boolean chk(String str) { ... }
private static final int N = 100;
public class Handler { public void handle(String p, String rng) { ... } }
```

[🔥 Violação e Explicação]: nomes genéricos (Handler, get, N, chk, p, rng) poluem a busca lexical, escondem a intenção e violam a regra "Don't Abbreviate". O agente tem que ler o corpo para descobrir o que o método faz. Se pesquisar pelo nome retorna coisas irrelevantes, o nome está ruim para a IA.

[✅ Exemplo de Refatoração]:

```
// nome diz o que faz; busca lexical (rg "AutorizacaoExpiradaHandler") cai direto
public List<Produto> buscarAtivosPorCategoria(String categoria) { ... }
public boolean precoEhValido(BigDecimal preco) { ... }
private static final int TAMANHO_MAXIMO_PAGINA = 100;
public class AutorizacaoExpiradaHandler {
    public void expirarAutorizacao(AutorizacaoId id, MotivoExpiracao motivo) { ... }
}
```

Nomes genéricos proibidos: Handler, Manager, Helper, Util, Data, Process, Info, Common, Base. Use nomes de domínio.

Imutabilidade

[❌ Código Não Aderente]:

```
// classe com setters publicos: estado mutavel depois da construcao
public class Market {
    private Long id;
    private String name;
    public void setId(Long id) { this.id = id; }
    public void setName(String name) { this.name = name; }
}
```

[🔥 Violação e Explicação]: setters públicos expõem o estado interno mutável; o chamador pode alterar id/name após a construção, quebrando invariantes do domínio e criando bugs sutis em fluxos assíncronos.

[✅ Exemplo de Refatoração]:

```
// record para DTOs e value objects (imutavel, equals/hashCode/toString gerados)
public record MarketDto(Long id, String name, MarketStatus status) {}

// ou classe com final fields e getters only
public class Market {
    private final Long id;
    private final String name;
    // getters only, no setters
}
```

Optional — uso correto

[❌ Código Não Aderente]:

```
public Market buscarPorSlug(String slug) {
    Optional<Market> market = marketRepository.findBySlug(slug);
    return market.get(); // NoSuchElementException se vazio
}
```

[🔥 Violação e Explicação]: Optional.get() sem .orElse().orElseThrow().isPresent() joga a decisão para erro em runtime sem chance do chamador reagir.

[✅ Exemplo de Refatoração]:

```
public Market buscarPorSlug(String slug) {
    return marketRepository.findBySlug(slug)
        .orElseThrow(() -> new EntityNotFoundException("Market not found: " + slug));
}
```

Streams — pipelines curtos, sem efeito colateral

[❌ Código Não Aderente]:

```
List<String> nomesAtivos = new ArrayList<>();
markets.stream().forEach(m -> {
    if (m.isAtivo()) {
        nomesAtivos.add(m.name().toUpperCase());
    }
});
```

[🔥 Violação e Explicação]: forEach capturando variável externa é anti-pattern clássico; força rastrear o efeito colateral e impede paralelização.

[✓ Exemplo de Refatoração]:

```
List<String> names = markets.stream()
    .filter(Market::isAtivo)
    .map(m -> m.name().toUpperCase())
    .toList();
```

Exception handling

- Use **unchecked exceptions** para erros de domínio (`BusinessException`).
- **Crie exceções específicas** em vez de `RuntimeException` genérica.
- **Evite** `catch (Exception ex) amplo`.
- **Sempre preserve a causa** (`throw new ApplicationException(msg, e)`).

[✗ Código Não Aderente]:

```
try {
    integracaoClient.enviar(pedido);
} catch (IOException e) {
    throw new RuntimeException(e.getMessage()); // Perde a causa (e)
}
```

[🔥 Violação e Explicação]: perde a `Throwable` cause (stack trace) e a mensagem original. Mensagem vaga força a IA a gastar turnos extras para descobrir o que deu errado.

[✓ Exemplo de Refatoração]:

```
try {
    integracaoClient.enviar(pedido);
} catch (IOException e) {
    throw new ApplicationException("Falha ao enviar pedido " + pedido.id(), e);
}
```

Genéricos e type safety (Tipos explícitos para IA)**[✗ Código Não Aderente]:**

```
public Map indexById(Collection items) { ... } // sem type safety
```

[🔥 Violação e Explicação]: código sem anotações de tipo ou com raw types obriga agentes e humanos a inferirem o que entra e sai, gerando falhas. O agente poupa trabalho de descoberta em códigos tipados.

[✓ Exemplo de Refatoração]:

```
public <T extends Identifiable> Map<Long, T> indexById(Collection<T> items) { ... }
```

Tell, Don't Ask & No Getters/Setters (Object Calisthenics)

O código cliente não deve perguntar o estado interno de um objeto para tomar decisões. A regra "No Getters/Setters/Properties" foca no encapsulamento de comportamentos e evita a exposição indevida que viola a orientação a objetos. Uma classe exposta à manipulação de estado por fora gera domínio anêmico e separa os dados do comportamento.

[✗ Código Não Aderente]:

```
// Dominio anemico: o objeto e um saco de dados, a acao e feita de fora
public void aplicarDesconto(Produto produto) {
    if (produto.getPreco() > 50) {
        produto.setPreco(produto.getPreco() - 10);
    }
}
```

[🔥 Violação e Explicação]:

1. Ferimento de encapsulamento e do princípio "Tell, Don't Ask". O cliente pergunta o preço para decidir a regra.
2. O agente LLM precisa ler 2 arquivos (`Produto` + `Service`) para entender a regra.
3. Se a regra mudar, será necessário caçar onde esse getter/setter foi usado no código.

[✓ Exemplo de Refatoração]:

```
public class Produto {
    private BigDecimal preco;

    // O objeto controla e protege sua propria regra
    public void aplicarDesconto(BigDecimal valorDesconto) {
        if (this.preco.compareTo(new BigDecimal("50")) > 0) {
```

```

        this.preco = this.preco.subtract(valorDesconto);
    }
}

// O cliente so envia o comando
public void processar(Produto produto) {
    produto.aplicarDesconto(new BigDecimal("10"));
}

```

Primitive Obsession & Wrap All Primitives (Object Calisthenics)

Não use tipos primitivos para representar conceitos com comportamento. A regra "Wrap All Primitives And Strings" determina que variáveis primitivas com comportamento específico de domínio (como validações próprias) devem ser encapsuladas em Objetos.

[❌ Código Não Aderente]:

```

public class Pessoa {
    private String cpf; // Obsessao por tipo primitivo

    public Pessoa(String cpf) {
        if (cpf == null || cpf.length() != 11) throw new IllegalArgumentException();
        this.cpf = cpf;
    }
}

```

[🔥 **Violação e Explicação**]: O CPF em formato String espalha lógica de validação. O agente de IA precisa inferir o formato, consumindo janela de contexto. A regra de validação não pertence estruturalmente à Pessoa.

[✅ Exemplo de Refatoração]:

```

public record Cpf(String numero) { // Value object imutavel que contem suas regras
    public Cpf {
        if (numero == null || numero.length() != 11) {
            throw new IllegalArgumentException("CPF Invalido");
        }
    }
}

public class Pessoa {
    private Cpf cpf; // Propriedade tipada e protegida
}

```

First Class Collections (Object Calisthenics)

Qualquer classe que contenha uma coleção não deve conter outras variáveis de membro. Se você tem um conjunto de elementos, crie uma classe dedicada exclusivamente a essa coleção.

[❌ Código Não Aderente]:

```

public class Empresa {
    private String razaoSocial;
    private List<Funcionario> funcionarios; // Colecao misturada com outros atributos

    // Metodos que manipulam a lista se misturam com metodos da empresa
    public List<Funcionario> buscarContadores() {
        return funcionarios.stream().filter(f -> f.getCargo().equals("contador")).toList();
    }
}

```

[🔥 **Violação e Explicação**]: Os comportamentos de filtro e agrupamento de funcionários poluem a classe Empresa.

[✅ Exemplo de Refatoração]:

```

public class QuadroFuncionarios { // First Class Collection
    private final List<Funcionario> funcionarios;

    public QuadroFuncionarios(List<Funcionario> funcionarios) {
        this.funcionarios = funcionarios;
    }

    // Comportamentos especificos tem um lar
    public List<Funcionario> buscarContadores() {
        return funcionarios.stream().filter(f -> f.getCargo().equals("contador")).toList();
    }
}

public class Empresa {
    private String razaoSocial;
}

```

```
private QuadroFuncionarios quadro;  
}
```

One Dot Per Line / Law of Demeter (Object Calisthenics)

Evite cadeias extensas de chamadas que atravessam vários objetos. Se você usa mais de um ponto na mesma linha, o seu objeto é um intermediário sabendo demais sobre a estrutura dos outros.

[❌ Código Não Aderente]:

```
// Navegando a estrutura interna (quebra de encapsulamento)  
String nomeChefe = funcionario.getDepartamento().getChefe().getNome();
```

[🔥 Violação e Explicação]: A estrutura interna do objeto fica exposta, dificultando a leitura e acoplando fortemente as classes.

[✅ Exemplo de Refatoração]:

```
// O objeto expressa intencoes via metodos comportamentais diretos  
String nomeChefe = funcionario.getNomeChefeDepartamento();
```

No Classes With More Than Two Instance Variables (Alta Coesão)

Classes não devem ter mais do que duas variáveis de instância, forçando um alto nível de coesão, composição e abstração. O objetivo prático é agrupar variáveis em lógicas menores quando a classe assume muitas responsabilidades.

[❌ Código Não Aderente]:

```
public class Funcionario {  
    private String nome;  
    private int idade;  
    private String cargo;  
    private String departamento;  
}
```

[🔥 Violação e Explicação]: A classe tem muitos atributos e assume informações tanto pessoais quanto contratuais.

[✅ Exemplo de Refatoração]:

```
public class Funcionario {  
    private InformacoesPessoais dadosPessoais; // Agrupa nome e idade  
    private InformacoesTrabalho dadosTrabalho; // Agrupa cargo e departamento  
}
```

Replace Magic Number with Symbolic Constant

Qualquer literal numérico ou `String` com **significado de domínio** é proibido no meio de validações. Deve virar constante nomeada.

[❌ Código Não Aderente]:

```
if (valor.compareTo(new BigDecimal("50000")) > 0) { ... }
```

[🔥 Violação e Explicação]: Magic Numbers escondem a regra. O agente precisa adivinhar o motivo da limitação.

[✅ Exemplo de Refatoração]:

```
// Constante documenta a regra  
private static final BigDecimal LIMITE_MAXIMO_PIX = new BigDecimal("50000");  
if (valor.compareTo(LIMITE_MAXIMO_PIX) > 0) { ... }
```

Guard Clauses & Don't Use "Else" (Object Calisthenics)

O uso de `else` é **proibido**. Assuma o fluxo padrão e faça validações através de Fail-Fast, Early Return ou Guard Clauses. Cada nível de indentação extra multiplica o custo cognitivo.

[❌ Código Não Aderente]:

```
public void processar(Pedido pedido) {  
    if (pedido != null) {  
        if (!pedido.getItems().isEmpty()) {  
            executar(pedido);  
        } else {  
            throw new BusinessException("sem itens");  
        }  
    }  
}
```

[🔥 Violação e Explicação]: Aninhamento de ifs aumenta a complexidade e polui a legibilidade. Para LLMs, o "else" desvia a atenção da lógica contínua.

[✓ Exemplo de Refatoração]:

```
public void processar(Pedido pedido) {
    if (pedido == null) return;

    if (pedido.getItems().isEmpty()) {
        throw new BusinessException("sem itens");
    }

    executar(pedido);
}
```

Clean Code for AI (Arquitetura para o Agente)

Regras vitais para quando o LLM interage e edita a base de código:

1. **Only One Level Of Indentation Per Method & Manter Métodos Pequenos:** Métodos curtos, contendo de 4 a 20 linhas ou 1 nível de indentação, cabem na mesma tool call do LLM, evitando perda de contexto.
2. **SRP (Responsabilidade Única):** Permite edições isoladas via AI sem efeito colateral destrutivo.
3. **Comentários de Proveniência:** Embora comentários óbvios (ex: `// incrementa i`) consumam tokens de IA à toa e devam sumir, documentações que explicam o *motivo* da decisão de negócio são cruciais.
4. **Testes que o agente consegue rodar:** Testes automatizados rápidos e headless (TDD) atuam como bússola, diferenciando o agente ágil do que trabalha "chutando".
5. **Injeção de Dependências:** Instanciar dependências hardcoded impede o isolamento no momento do teste. Usar DI facilita a refatoração e as suítes de teste.

Refactorings do Fowler e Bloaters — guia rápido

Além dos princípios, usamos técnicas do catálogo Refactoring Guru para sanar "Smells".

Remove Parameter**[✗ Código Não Aderente]:**

```
// "isCloud" e recebido mas nunca influencia o resultado
public Backend selecionarBackend(long tableId, ConnectContext context, boolean isCloud) { ... }
```

[🔥 Violação e Explicação]: Parâmetro morto polui a assinatura e é uma abstração especulativa.

[✓ Exemplo de Refatoração]:

```
public Backend selecionarBackend(long tableId, ConnectContext context) { ... }
```

Extract Method

Resolvendo o problema do *Long Method* (Bloater).

[✗ Código Não Aderente]:

```
public void processar(Pedido pedido) {
    // 20 linhas de validacao aqui
    // logica principal
}
```

[🔥 Violação e Explicação]: Validações inline espessas dificultam ler o fluxo real de domínio. O agente perde o foco de qual ação principal é realizada.

[✓ Exemplo de Refatoração]:

```
public void processar(Pedido pedido) {
    validarPedido(pedido);
    // logica principal
}
```

Replace Conditional with Polymorphism

Resolvendo o cheiro de *Switch Statements* ou longos *ifs* encadeados que checam tipos.

[✗ Código Não Aderente]:

```
if (pagamento instanceof Pix) return BigDecimal.ZERO;
if (pagamento instanceof Cartao) return BigDecimal.valueOf(0.03);
throw new IllegalStateException("Tipo desconhecido");
```


[🔥 **Violação e Explicação**]: Não é extensível, é aberto a falhas em runtime ao adicionar novo tipo.

[✅ **Exemplo de Refatoração**]:

```
// Pattern matching / switch exaustivo moderno
return switch (pagamento) {
    case Pix p      -> BigDecimal.ZERO;
    case Cartao c   -> BigDecimal.valueOf(0.03);
};
```

Introduce Parameter Object

Trata *Data Clumps* onde o mesmo conjunto de parâmetros é copiado entre métodos.

[❌ **Código Não Aderente**]:

```
public void buscar(LocalDate inicio, LocalDate fim, String status, int pagina) { ... }
```

[🔥 **Violação e Explicação**]: Propensão a erros ao alterar assinatura; dados andam juntos mas não formam um objeto.

[✅ **Exemplo de Refatoração**]:

```
public record FiltroPedido(LocalDate inicio, LocalDate fim, String status) {}
public void buscar(FiltroPedido filtro, int pagina) { ... }
```

Quem aplica o quê

Situação	Quem	Skill
Aplicar refactoring em uma classe/método	sessão principal	esta skill
Revisar diff/PR com checklist de severidade	agent java-revisor	revisao-de-codigo-java
Remoção de parâmetro focada (passo-a-passo)	sessão principal	refactoring-remove-parameter
Limpar imports não usados	sessão principal	remover-imports-nao-usados
Centralizar configuração dispersa (Shotgun Surgery)	session/engenheiro-devops	java-architecture
Decidir onde mora um value object novo	session/java-construtor	arquitetura-limpa-java

```
with open("SKILL_v1.2.0.md", "w", encoding="utf-8") as f: f.write(content)

print(f"File created successfully at {os.path.abspath('SKILL_v1.2.0.md')}")
```

```
```python?code_reference&code_event_index=7
import os

content = """---
name: qualidade-codigo-java
description: "Application-side guide for clean code in Java – DRY, KISS, YAGNI, naming, immutability, `Optional`, st
license: MIT
metadata:
 author: https://github.com/srportto/srportto
 co-author: https://github.com/Jeffallan/claude-skills
 version: "1.2.0"
 domain: code-quality
 triggers: clean code, boas práticas, refatorar, DRY, KISS, YAGNI, imutabilidade, Optional, streams, Fowler, Object
 role: reference
 scope: code-quality
 output-format: code
 related-skills: revisao-de-codigo-java, padroes-de-projeto-java, refatorador-java, java-moderno

Qualidade de Código Java (clean code + refactoring)

Visão geral

Guia de **aplicação** de clean code em Java – DRY, KISS, YAGNI, nomenclatura, imutabilidade, `Optional`, streams, ex

Quando NÃO usar: para revisar um diff/PR com checklist por severidade, use `revisao-de-codigo-java` (ela referen

Clean code – princípios com exemplo
```

> **Coesão com `revisao-de-codigo-java`:** esta skill é o "lado ativo" (o **como** aplicar cada refactoring). A `rev`

> **Princípio-mestre (Clean Code for AI):** além de bom para humanos, todo código deste catálogo deve estar **otimiz**

### DRY — Don't Repeat Yourself

**[❌ Código Não Aderente]:**

```
```java
// logica de validacao duplicada em dois metodos
public void criarUsuario(UsuarioRequest req) {
    if (req.getEmail() == null || !req.getEmail().contains("@")) {
        throw new ValidationException("Email invalido");
    }
}

public void atualizarUsuario(UsuarioRequest req) {
    if (req.getEmail() == null || !req.getEmail().contains("@")) {
        throw new ValidationException("Email invalido");
    }
}
```

[🔥 Violação e Explicação]: mesma validação em 2 lugares — a 3ª ocorrência (em `importarEmLote`, por exemplo) confirma o padrão. Manter a duplicação significa N lugares para corrigir quando a regra mudar.

[✅ Exemplo de Refatoração]:

```
// fonte unica: metodo privado resolve sem criar interface/factory para o futuro
public class UsuarioService {
    public void criarUsuario(UsuarioRequest req) { validarEmail(req.getEmail()); /* ... */ }
    public void atualizarUsuario(UsuarioRequest req) { validarEmail(req.getEmail()); /* ... */ }

    private void validarEmail(String email) {
        if (email == null || !email.contains("@")) {
            throw new ValidationException("Email invalido");
        }
    }
}
```

KISS — Keep It Simple / YAGNI — You Aren't Gonna Need It

[❌ Código Não Aderente]:

```
// sobre-engenharia para 1 implementacao, sem segunda variacao a vista
public interface UserFactory {
    User createUser();
}

public class ConcreteUserFactory implements UserFactory {
    public User createUser() { return new User(); }
}
```

[🔥 Violação e Explicação]: interface + implementação única **"para o futuro"** é a abstração especulativa clássica (YAGNI). O custo (mais arquivos para ler, mais para o agente raciocinar) não traz benefício enquanto houver 1 variante.

[✅ Exemplo de Refatoração]:

```
// chamada direta; implemente a abstracao quando a segunda variacao aparecer de fato
public User createUser() { return new User(); }
```

Convenções de nomenclatura (Object Calisthenics: Don't Abbreviate)

A regra do Object Calisthenics orienta a nunca usar nomes abreviados, pois nomes com significado ajudam no entendimento e previnem falhas de design.

```
// Classes/Records: PascalCase
public class MarketService {}
public record Money(BigDecimal amount, Currency currency) {}

// Metodos/campos: camelCase
private final MarketRepository marketRepository;
public Market findBySlug(String slug) {}

// Constantes: UPPER_SNAKE_CASE
private static final int MAX_PAGE_SIZE = 100;
```

Nomes que revelam intenção e são grepáveis:

[❌ Código Não Aderente]:

```
// abreviaco escuras e nomes genericos nao sao grepaveis
public List<Produto> get(String s) { ... }
```

```
public boolean chk(String str) { ... }
private static final int N = 100;
public class Handler { public void handle(String p, String rng) { ... } }
```

[🚩 Violação e Explicação]: nomes genéricos (Handler, get, N, chk, p, rng) poluem a busca lexical, escondem a intenção e violam a regra "Don't Abbreviate". O agente tem que ler o corpo para descobrir o que o método faz. Se pesquisar pelo nome retorna coisas irrelevantes, o nome está ruim para a IA.

[✅ Exemplo de Refatoração]:

```
// nome diz o que faz; busca lexical (rg "AutorizacaoExpiradaHandler") cai direto
public List<Produto> buscarAtivosPorCategoria(String categoria) { ... }
public boolean precoEhValido(BigDecimal preco) { ... }
private static final int TAMANHO_MAXIMO_PAGINA = 100;
public class AutorizacaoExpiradaHandler {
    public void expirarAutorizacao(AutorizacaoId id, MotivoExpiracao motivo) { ... }
}
```

Nomes genéricos proibidos: Handler, Manager, Helper, Util, Data, Process, Info, Common, Base. Use nomes de domínio.

Imutabilidade

[❌ Código Não Aderente]:

```
// classe com setters publicos: estado mutavel depois da construcao
public class Market {
    private Long id;
    private String name;
    public void setId(Long id) { this.id = id; }
    public void setName(String name) { this.name = name; }
}
```

[🚩 Violação e Explicação]: setters públicos expõem o estado interno mutável; o chamador pode alterar id/name após a construção, quebrando invariantes do domínio e criando bugs sutis em fluxos assíncronos.

[✅ Exemplo de Refatoração]:

```
// record para DTOs e value objects (imutavel, equals/hashCode/toString gerados)
public record MarketDto(Long id, String name, MarketStatus status) {}

// ou classe com final fields e getters only
public class Market {
    private final Long id;
    private final String name;
    // getters only, no setters
}
```

Optional — uso correto

[❌ Código Não Aderente]:

```
public Market buscarPorSlug(String slug) {
    Optional<Market> market = marketRepository.findBySlug(slug);
    return market.get(); // NoSuchElementException se vazio
}
```

[🚩 Violação e Explicação]: Optional.get() sem .orElse().orElseThrow().isPresent() joga a decisão para erro em runtime sem chance do chamador reagir.

[✅ Exemplo de Refatoração]:

```
public Market buscarPorSlug(String slug) {
    return marketRepository.findBySlug(slug)
        .orElseThrow(() -> new EntityNotFoundException("Market not found: " + slug));
}
```

Streams — pipelines curtos, sem efeito colateral

[❌ Código Não Aderente]:

```
List<String> nomesAtivos = new ArrayList<>();
markets.stream().forEach(m -> {
    if (m.isAtivo()) {
        nomesAtivos.add(m.name().toUpperCase());
    }
});
```

[🚩 Violação e Explicação]: forEach capturando variável externa é anti-pattern clássico; força rastrear o efeito colateral e impede paralelização.

[✓ Exemplo de Refatoração]:

```
List<String> names = markets.stream()
    .filter(Market::isAtivo)
    .map(m -> m.name().toUpperCase())
    .toList();
```

Exception handling

- Use **unchecked exceptions** para erros de domínio (`BusinessException`).
- **Crie exceções específicas** em vez de `RuntimeException` genérica.
- **Evite** `catch (Exception ex)` amplo.
- **Sempre preserve a causa** (`throw new ApplicationException(msg, e)`).

[✗ Código Não Aderente]:

```
try {
    integracaoClient.enviar(pedido);
} catch (IOException e) {
    throw new RuntimeException(e.getMessage()); // Perde a causa (e)
}
```

[🔥 Violação e Explicação]: perde a `Throwable` `cause` (stack trace) e a mensagem original. Mensagem vaga força a IA a gastar turnos extras para descobrir o que deu errado.

[✓ Exemplo de Refatoração]:

```
try {
    integracaoClient.enviar(pedido);
} catch (IOException e) {
    throw new ApplicationException("Falha ao enviar pedido " + pedido.id(), e);
}
```

Genéricos e type safety (Tipos explícitos para IA)**[✗ Código Não Aderente]:**

```
public Map indexById(Collection items) { ... } // sem type safety
```

[🔥 Violação e Explicação]: código sem anotações de tipo ou com raw types obriga agentes e humanos a inferirem o que entra e sai, gerando falhas. O agente poupa trabalho de descoberta em códigos tipados.

[✓ Exemplo de Refatoração]:

```
public <T extends Identifiable> Map<Long, T> indexById(Collection<T> items) { ... }
```

Tell, Don't Ask & No Getters/Setters (Object Calisthenics)

O código cliente não deve perguntar o estado interno de um objeto para tomar decisões. A regra "No Getters/Setters/Properties" foca no encapsulamento de comportamentos e evita a exposição indevida que viola a orientação a objetos. Uma classe exposta à manipulação de estado por fora gera domínio anêmico e separa os dados do comportamento.

[✗ Código Não Aderente]:

```
// Dominio anemico: o objeto e saco de dados, a acao e feita de fora
public void aplicarDesconto(Produto produto) {
    if (produto.getPreco() > 50) {
        produto.setPreco(produto.getPreco() - 10);
    }
}
```

[🔥 Violação e Explicação]:

1. Ferimento de encapsulamento e do princípio "Tell, Don't Ask". O cliente pergunta o preço para decidir a regra.
2. O agente LLM precisa ler 2 arquivos (`Produto` + `Service`) para entender a regra.
3. Se a regra mudar, será necessário caçar onde esse getter/setter foi usado no código.

[✓ Exemplo de Refatoração]:

```
public class Produto {
    private BigDecimal preco;

    // O objeto controla e protege sua propria regra
    public void aplicarDesconto(BigDecimal valorDesconto) {
        if (this.preco.compareTo(new BigDecimal("50")) > 0) {
```

```

        this.preco = this.preco.subtract(valorDesconto);
    }
}

// O cliente so envia o comando
public void processar(Produto produto) {
    produto.aplicarDesconto(new BigDecimal("10"));
}

```

Primitive Obsession & Wrap All Primitives (Object Calisthenics)

Não use tipos primitivos para representar conceitos com comportamento. A regra "Wrap All Primitives And Strings" determina que variáveis primitivas com comportamento específico de domínio (como validações próprias) devem ser encapsuladas em Objetos.

[❌ Código Não Aderente]:

```

public class Pessoa {
    private String cpf; // Obsessao por tipo primitivo

    public Pessoa(String cpf) {
        if (cpf == null || cpf.length() != 11) throw new IllegalArgumentException();
        this.cpf = cpf;
    }
}

```

[🔥 **Violação e Explicação**]: O CPF em formato String espalha lógica de validação. O agente de IA precisa inferir o formato, consumindo janela de contexto. A regra de validação não pertence estruturalmente à Pessoa.

[✅ Exemplo de Refatoração]:

```

public record Cpf(String numero) { // Value object imutavel que contem suas regras
    public Cpf {
        if (numero == null || numero.length() != 11) {
            throw new IllegalArgumentException("CPF Invalido");
        }
    }
}

public class Pessoa {
    private Cpf cpf; // Propriedade tipada e protegida
}

```

First Class Collections (Object Calisthenics)

Qualquer classe que contenha uma coleção não deve conter outras variáveis de membro. Se você tem um conjunto de elementos, crie uma classe dedicada exclusivamente a essa coleção.

[❌ Código Não Aderente]:

```

public class Empresa {
    private String razaoSocial;
    private List<Funcionario> funcionarios; // Colecao misturada com outros atributos

    // Metodos que manipulam a lista se misturam com metodos da empresa
    public List<Funcionario> buscarContadores() {
        return funcionarios.stream().filter(f -> f.getCargo().equals("contador")).toList();
    }
}

```

[🔥 **Violação e Explicação**]: Os comportamentos de filtro e agrupamento de funcionários poluem a classe Empresa.

[✅ Exemplo de Refatoração]:

```

public class QuadroFuncionarios { // First Class Collection
    private final List<Funcionario> funcionarios;

    public QuadroFuncionarios(List<Funcionario> funcionarios) {
        this.funcionarios = funcionarios;
    }

    // Comportamentos especificos tem um lar
    public List<Funcionario> buscarContadores() {
        return funcionarios.stream().filter(f -> f.getCargo().equals("contador")).toList();
    }
}

public class Empresa {
    private String razaoSocial;
}

```

```
private QuadroFuncionarios quadro;  
}
```

One Dot Per Line / Law of Demeter (Object Calisthenics)

Evite cadeias extensas de chamadas que atravessam vários objetos. Se você usa mais de um ponto na mesma linha, o seu objeto é um intermediário sabendo demais sobre a estrutura dos outros.

[❌ Código Não Aderente]:

```
// Navegando a estrutura interna (quebra de encapsulamento)  
String nomeChefe = funcionario.getDepartamento().getChefe().getNome();
```

[🔥 Violação e Explicação]: A estrutura interna do objeto fica exposta, dificultando a leitura e acoplando fortemente as classes.

[✅ Exemplo de Refatoração]:

```
// O objeto expressa intencoes via metodos comportamentais diretos  
String nomeChefe = funcionario.getNomeChefeDepartamento();
```

No Classes With More Than Two Instance Variables (Alta Coesão)

Classes não devem ter mais do que duas variáveis de instância, forçando um alto nível de coesão, composição e abstração. O objetivo prático é agrupar variáveis em lógicas menores quando a classe assume muitas responsabilidades.

[❌ Código Não Aderente]:

```
public class Funcionario {  
    private String nome;  
    private int idade;  
    private String cargo;  
    private String departamento;  
}
```

[🔥 Violação e Explicação]: A classe tem muitos atributos e assume informações tanto pessoais quanto contratuais.

[✅ Exemplo de Refatoração]:

```
public class Funcionario {  
    private InformacoesPessoais dadosPessoais; // Agrupa nome e idade  
    private InformacoesTrabalho dadosTrabalho; // Agrupa cargo e departamento  
}
```

Replace Magic Number with Symbolic Constant

Qualquer literal numérico ou `String` com **significado de domínio** é proibido no meio de validações. Deve virar constante nomeada.

[❌ Código Não Aderente]:

```
if (valor.compareTo(new BigDecimal("50000")) > 0) { ... }
```

[🔥 Violação e Explicação]: Magic Numbers escondem a regra. O agente precisa adivinhar o motivo da limitação.

[✅ Exemplo de Refatoração]:

```
// Constante documenta a regra  
private static final BigDecimal LIMITE_MAXIMO_PIX = new BigDecimal("50000");  
if (valor.compareTo(LIMITE_MAXIMO_PIX) > 0) { ... }
```

Guard Clauses & Don't Use "Else" (Object Calisthenics)

O uso de `else` é **proibido**. Assuma o fluxo padrão e faça validações através de Fail-Fast, Early Return ou Guard Clauses. Cada nível de indentação extra multiplica o custo cognitivo.

[❌ Código Não Aderente]:

```
public void processar(Pedido pedido) {  
    if (pedido != null) {  
        if (!pedido.getItems().isEmpty()) {  
            executar(pedido);  
        } else {  
            throw new BusinessException("sem itens");  
        }  
    }  
}
```

[🔥 Violação e Explicação]: Aninhamento de ifs aumenta a complexidade e polui a legibilidade. Para LLMs, o "else" desvia a atenção da lógica contínua.

[✓ Exemplo de Refatoração]:

```
public void processar(Pedido pedido) {
    if (pedido == null) return;

    if (pedido.getItems().isEmpty()) {
        throw new BusinessException("sem itens");
    }

    executar(pedido);
}
```

Clean Code for AI (Arquitetura para o Agente)

Regras vitais para quando o LLM interage e edita a base de código:

1. **Only One Level Of Indentation Per Method & Manter Métodos Pequenos:** Métodos curtos, contendo de 4 a 20 linhas ou 1 nível de indentação, cabem na mesma tool call do LLM, evitando perda de contexto.
2. **SRP (Responsabilidade Única):** Permite edições isoladas via AI sem efeito colateral destrutivo.
3. **Comentários de Proveniência:** Embora comentários óbvios (ex: `// incrementa i`) consumam tokens de IA à toa e devam sumir, documentações que explicam o *motivo* da decisão de negócio são cruciais.
4. **Testes que o agente consegue rodar:** Testes automatizados rápidos e headless (TDD) atuam como bússola, diferenciando o agente ágil do que trabalha "chutando".
5. **Injeção de Dependências:** Instanciar dependências hardcoded impede o isolamento no momento do teste. Usar DI facilita a refatoração e as suítes de teste.

Refactorings do Fowler e Bloaters — guia rápido

Além dos princípios, usamos técnicas do catálogo Refactoring Guru para sanar "Smells".

Remove Parameter**[✗ Código Não Aderente]:**

```
// "isCloud" e recebido mas nunca influencia o resultado
public Backend selecionarBackend(long tableId, ConnectContext context, boolean isCloud) { ... }
```

[🔥 Violação e Explicação]: Parâmetro morto polui a assinatura e é uma abstração especulativa.

[✓ Exemplo de Refatoração]:

```
public Backend selecionarBackend(long tableId, ConnectContext context) { ... }
```

Extract Method

Resolvendo o problema do *Long Method* (Bloater).

[✗ Código Não Aderente]:

```
public void processar(Pedido pedido) {
    // 20 linhas de validacao aqui
    // logica principal
}
```

[🔥 Violação e Explicação]: Validações inline espessas dificultam ler o fluxo real de domínio. O agente perde o foco de qual ação principal é realizada.

[✓ Exemplo de Refatoração]:

```
public void processar(Pedido pedido) {
    validarPedido(pedido);
    // logica principal
}
```

Replace Conditional with Polymorphism

Resolvendo o cheiro de *Switch Statements* ou longos *ifs* encadeados que checam tipos.

[✗ Código Não Aderente]:

```
if (pagamento instanceof Pix) return BigDecimal.ZERO;
if (pagamento instanceof Cartao) return BigDecimal.valueOf(0.03);
throw new IllegalStateException("Tipo desconhecido");
```

[🔥 **Violação e Explicação**]: Não é extensível, é aberto a falhas em runtime ao adicionar novo tipo.

[✅ **Exemplo de Refatoração**]:

```
// Pattern matching / switch exaustivo moderno
return switch (pagamento) {
    case Pix p      -> BigDecimal.ZERO;
    case Cartao c   -> BigDecimal.valueOf(0.03);
};
```

Introduce Parameter Object

Trata *Data Clumps* onde o mesmo conjunto de parâmetros é copiado entre métodos.

[❌ **Código Não Aderente**]:

```
public void buscar(LocalDate inicio, LocalDate fim, String status, int pagina) { ... }
```

[🔥 **Violação e Explicação**]: Propensão a erros ao alterar assinatura; dados andam juntos mas não formam um objeto.

[✅ **Exemplo de Refatoração**]:

```
public record FiltroPedido(LocalDate inicio, LocalDate fim, String status) {}
public void buscar(FiltroPedido filtro, int pagina) { ... }
```

Quem aplica o quê

Situação	Quem	Skill
Aplicar refactoring em uma classe/método	sessão principal	esta skill
Revisar diff/PR com checklist de severidade	agent java-revisor	revisao-de-codigo-java
Remoção de parâmetro focada (passo-a-passo)	sessão principal	refactoring-remove-parameter
Limpar imports não usados	sessão principal	remover-imports-nao-usados
Centralizar configuração dispersa (Shotgun Surgery)	session/engenheiro-devops	java-architecture
Decidir onde mora um value object novo	session/java-construtor	arquitetura-limpa-java
""		

```
with open("SKILL_v1.2.0.md", "w", encoding="utf-8") as f: f.write(content)

print(f"File created successfully at {os.path.abspath('SKILL_v1.2.0.md')}")
```

I have generated the requested document and saved it directly to your Google Drive.

```
[file-tag: drive-uploaded-file-13Bcc_26AZFrd4_aJHVQdJ9dTbb0xvYgcxaYlyUarpE]
```

User prompt: agora compile tudo em um unico arquivo, idem o anexo e me permita fazer o download

Response: Aqui está o arquivo completo e unificado do guia **Qualidade de Código Java (clean code + refactoring)**, incluindo todo o cabeçalho YAML de metadados e o conteúdo técnico estruturado.

Você pode copiar o bloco de código abaixo e salvá-lo com o nome `qualidade-codigo-java.md`:

```
---
name: qualidade-codigo-java
description: "Application-side guide for clean code in Java – DRY, KISS, YAGNI, naming, immutability, `Optional`, st
license: MIT
metadata:
  author: https://github.com/srportto/srportto
  co-author: https://github.com/Jeffallan/claude-skills
  version: "1.1.0"
  domain: code-quality
  triggers: clean code, boas práticas, refatorar, DRY, KISS, YAGNI, imutabilidade, Optional, streams, Fowler
  role: reference
  scope: code-quality
  output-format: code
  related-skills: revisao-de-codigo-java, padroes-de-projeto-java, refatorador-java, java-moderno
---

# Qualidade de Código Java (clean code + refactoring)
```


Visão geral

Guia de ****aplicação**** de clean code em Java – DRY, KISS, YAGNI, nomenclatura, imutabilidade, `Optional`, streams, exception handling – e de refactorings do catálogo do Fowler (Remove Parameter, Extract Method, Replace Magic Number, etc.). Esta skill é o "lado ativo" da revisão: a `revisao-de-codigo-java` diz ****o que revisar**** com checklist e severidades; esta skill diz ****como aplicar**** o que a revisão aponta.

****Quando NÃO usar:**** para revisar um diff/PR com checklist por severidade, use `revisao-de-codigo-java` (ela referencia esta aqui). Para a regra de dependência entre camadas (`domain`/`application`/`infrastructure`), use `arquitetura-limpa-java`. Para JPA/Hibernate (N+1, dirty checking), use `persistencia-jpa`. Para logging (formato, MDC), use `padrao-de-logs-java`.

Clean code – princípios com exemplo

> ****Coesão com `revisao-de-codigo-java`:**** esta skill é o "lado ativo" (o ****como**** aplicar cada > refactoring). A `revisao-de-codigo-java` é o "lado passivo" (o ****que**** revisar com checklist > e severidades). Mesmo formato de exemplo / / , mesmas terminologias > (`Magic Number`, `Primitive Obsession`, `Guard Clause`, `Tell Don't Ask`).

> ****Princípio-mestre (Clean Code for AI):**** além de bom para humanos, todo código deste > catálogo deve estar ****otimizado para a janela de contexto do LLM**** – nomes grepáveis, > métodos curtos, arquivos pequenos, tipos explícitos e comentários "por que". Cada seção > abaixo reforça esse objetivo.

DRY – Don't Repeat Yourself

```
**[ Código Não Aderente]:**
```java
// logica de validacao duplicada em dois metodos
public void criarUsuario(UsuarioRequest req) {
 if (req.getEmail() == null || !req.getEmail().contains("@")) {
 throw new ValidationException("Email invalido");
 }
}

public void atualizarUsuario(UsuarioRequest req) {
 if (req.getEmail() == null || !req.getEmail().contains("@")) {
 throw new ValidationException("Email invalido");
 }
}
}
```

**[ Violação e Explicação]:** mesma validação em 2 lugares — a 3ª ocorrência (em `importarEmLote`, por exemplo) confirma o padrão. Manter a duplicação significa N lugares para corrigir quando a regra mudar.

### **[ Exemplo de Refatoração]:**

```
// fonte unica: metodo privado resolve sem criar interface/factory para o futuro
public class UsuarioService {
 public void criarUsuario(UsuarioRequest req) { validarEmail(req.getEmail()); /* ... */ }
 public void atualizarUsuario(UsuarioRequest req) { validarEmail(req.getEmail()); /* ... */ }

 private void validarEmail(String email) {
 if (email == null || !email.contains("@")) {
 throw new ValidationException("Email invalido");
 }
 }
}
}
```

**DRY com bom senso:** regra das 3 ocorrências — na 1ª e 2ª, duplicar pode ser mais barato que a abstração errada; extraia na 3ª. Não crie `EmailValidator` com interface e implementação única "para o futuro" — abstração especulativa é over-engineering (ver `padroes-de-projeto-java`, seção "Quando NÃO aplicar pattern").

## KISS — Keep It Simple / YAGNI — You Aren't Gonna Need It

### **[ Código Não Aderente]:**

```
// sobre-engenharia para 1 implementacao, sem segunda variacao a vista
public interface UserFactory {
 User createUser();
}

public class ConcreteUserFactory implements UserFactory {
 public User createUser() { return new User(); }
}
}
```

**[ Violação e Explicação]:** interface + implementação única **"para o futuro"** é a abstração especulativa clássica (YAGNI). O custo (mais arquivos para ler, mais para o agente raciocinar) não traz benefício enquanto houver 1 variante.

### **[ Exemplo de Refatoração]:**

```
// chamada direta; implemente a abstracao quando a segunda variacao aparecer de fato
public User createUser() { return new User(); }
```

## Convenções de nomenclatura

```
// Classes/Records: PascalCase
public class MarketService {}
public record Money(BigDecimal amount, Currency currency) {}
```

```
// Metodos/campos: camelCase
private final MarketRepository marketRepository;
public Market findBySlug(String slug) {}
```

```
// Constantes: UPPER_SNAKE_CASE
private static final int MAX_PAGE_SIZE = 100;
```

**Nomes que revelam intenção e são grepáveis** (não abrevie sem motivo):

[❌ **Código Não Aderente**]:

```
// abreviaco es obscuras e nomes genericos nao sao grepaveis
public List<Produto> get(String s) { ... }
public boolean chk(String str) { ... }
private static final int N = 100;
public class Handler { public void handle(String p, String rng) { ... } }
```

[🔥 **Violação e Explicação**]: nomes genéricos (Handler, get, N, chk, p, rng) poluem a busca lexical e escondem a intenção. O agente tem que ler o corpo para descobrir o que o método faz.

[✅ **Exemplo de Refatoração**]:

```
// nome diz o que faz; busca lexical (rg "AutorizacaoExpiradaHandler") cai direto
public List<Produto> buscarAtivosPorCategoria(String categoria) { ... }
public boolean precoEhValido(BigDecimal preco) { ... }
private static final int TAMANHO_MAXIMO_PAGINA = 100;
public class AutorizacaoExpiradaHandler {
 public void expirarAutorizacao(AutorizacaoId id, MotivoExpiracao motivo) { ... }
}
```

**Nomes genéricos proibidos** (poluem grep, escondem intenção): Handler, Manager, Helper, Util, Data, Process, Info, Common, Base. Use nomes de domínio. Exceção: Manager é aceitável **quando** o domínio é o próprio gerenciado (PixBufferRingPartitionPurgeManager), nunca sozinho.

Use português ou inglês consistentemente dentro do mesmo pacote/classe — não misture.

## Imutabilidade

[❌ **Código Não Aderente**]:

```
// classe com setters publicos: estado mutavel depois da construo
public class Market {
 private Long id;
 private String name;
 public void setId(Long id) { this.id = id; }
 public void setName(String name) { this.name = name; }
}
```

[🔥 **Violação e Explicação**]: setters públicos expõem o estado interno mutável; o chamador pode alterar id/name após a construção, quebrando invariantes do domínio e criando bugs sutis em fluxos assíncronos.

[✅ **Exemplo de Refatoração**]:

```
// record para DTOs e value objects (imutavel, equals/hashCode/toString gerados)
public record MarketDto(Long id, String name, MarketStatus status) {}

// ou classe com final fields e getters only
public class Market {
 private final Long id;
 private final String name;
 // getters only, no setters
}
```

Records são o padrão deste catálogo (ver java-moderno): use para DTOs, value objects, chaves compostas (IdAutorizacao). Não use records quando precisar de mutabilidade ou herança.

## Optional — uso correto

[❌ **Código Não Aderente**]:

```
// get() sem verificar presença
public Market buscarPorSlug(String slug) {
 Optional<Market> market = marketRepository.findBySlug(slug);
 return market.get(); // NoSuchElementException se vazio
}
```

**[🚨 Violação e Explicação]:** `Optional.get()` sem `.orElse().orElseThrow().isPresent()` joga a decisão para o `NoSuchElementException` em runtime; o caller não tem como reagir.

**[✅ Exemplo de Refatoração]:**

```
// retorne Optional de metodos find*, use map/flatMap em vez de get() direto
public Market buscarPorSlug(String slug) {
 return marketRepository.findBySlug(slug)
 .orElseThrow(() -> new EntityNotFoundException("Market not found: " + slug));
}
```

## Streams — pipelines curtos, sem efeito colateral

**[❌ Código Não Aderente]:**

```
// forEach com mutacao de lista externa
List<String> nomesAtivos = new ArrayList<>();
markets.stream().forEach(m -> {
 if (m.isAtivo()) {
 nomesAtivos.add(m.name().toUpperCase());
 }
});
```

**[🚨 Violação e Explicação]:** `forEach` capturando variável externa é o anti-pattern clássico de stream; força o agente a rastrear o efeito colateral e quebra paralelização futura.

**[✅ Exemplo de Refatoração]:**

```
// pipeline curto, transformacao pura
List<String> names = markets.stream()
 .filter(Market::isAtivo)
 .map(m -> m.name().toUpperCase())
 .toList();
```

Quando o pipeline exigiria múltiplos `flatMap`/estado acumulado só para simular um `for`, prefira o loop explícito — clareza vale mais que "tudo em stream".

## Exception handling

- Use **unchecked exceptions** para erros de domínio (`BusinessException` — mapeada para 422 pelo handler central; ver arquitetura-limpa-java).
- Crie exceções específicas do domínio (`MarketNotFoundException`) em vez de `RuntimeException` genérica.
- Evite `catch (Exception ex)` amplo, a menos que seja para relançar/logar centralmente.
- Sempre preserve a causa (`throw new ApplicationException(msg, e)`) — perder a stack trace original torna investigação quase impossível.
- Recursos — sempre `try-with-resources`; `close()` manual não executa se o código anterior lançar.

**[❌ Código Não Aderente]:**

```
// perde a causa
try {
 integracaoClient.enviar(pedido);
} catch (IOException e) {
 throw new RuntimeException(e.getMessage());
}
```

**[🚨 Violação e Explicação]:** perde a `Throwable` cause (stack trace original) e produz mensagem genérica sem contexto da operação; investigação quase impossível depois.

**[✅ Exemplo de Refatoração]:**

```
// especifica, com causa preservada
try {
 integracaoClient.enviar(pedido);
} catch (IOException e) {
 throw new ApplicationException("Falha ao enviar pedido " + pedido.id() + " para integracao", e);
}
```

## Genéricos e type safety

**[❌ Código Não Aderente]:**

```
// raw type
public Map indexById(Collection items) { ... } // sem type safety
```

**[🔥 Violação e Explicação]:** raw types desativam o type checker; o agente precisa inferir tipos a cada leitura e não recebe proteção contra `ClassCastException` em runtime.

**[✅ Exemplo de Refatoração]:**

```
// generic explicito
public <T Identifiable extends> Map<Long, T> indexById(Collection<T> items) { ... }
```

**Tell, Don't Ask (sem domínio anêmico)**

O código cliente **não deve** perguntar o estado interno de um objeto para tomar uma decisão por ele — a própria classe deve expor **métodos comportamentais** que realizam a ação com seus próprios dados. Getters/setters em entidades e value objects de domínio produzem **domínio anêmico**: as regras ficam espalhadas no service e a classe vira só um saco de dados.

**Quando NÃO aplicar:** DTOs de borda (request/response HTTP, mensagens de fila) **precisam** de getters para serialização Jackson/Avro. A regra vale para entidades de domínio e value objects. Ver `java-moderno` seção "Records" para quando usar record vs classe cheia.

**[❌ Código Não Aderente]:**

```
// Servico puxa saldo, faz matematica externa e devolve o resultado: dominio anemico
public void processarSaque(Conta conta, BigDecimal valor) {
 if (conta.getSaldo().compareTo(valor) >= 0) {
 conta.setSaldo(conta.getSaldo().subtract(valor));
 } else {
 throw new BusinessException("saldo insuficiente");
 }
}
```

**[🔥 Violação e Explicação]:**

1. **Object-Orientation Abuser** (Refactoring Guru): a classe `Conta` é um saco de dados; toda a regra "saque" vive no `Service`, espalhada.
2. **Concorrência**: dois saques simultâneos podem ler o mesmo saldo e terminar em inconsistência (lost update) — encapsular permite usar lock otimista dentro de `Conta`.
3. **Janela de contexto do LLM**: o agente precisa ler 2 arquivos (`Conta` + `Service`) para entender uma única regra; encapsular reduz para 1.

**[✅ Exemplo de Refatoração]:**

```
public class Conta {
 private BigDecimal saldo;
 // ... outros campos, ctor, equals, hashCode

 public void sacar(BigDecimal valor) {
 if (saldo.compareTo(valor) < 0) {
 throw new BusinessException("saldo insuficiente");
 }
 this.saldo = saldo.subtract(valor);
 }
}

// Service apenas delega; regra de negocio vive onde os dados vivem
public void processarSaque(Conta conta, BigDecimal valor) {
 conta.sacar(valor);
}
```

**Primitive Obsession (encapsular em value objects)**

Não use tipos primitivos (`long`, `int`, `String`, `double`, `BigDecimal` solto) para representar conceitos com comportamento, validação ou semântica próprios. Encapsule em records (value objects imutáveis) que carregam parsing, validação e operações.

**Exemplos de encapsulamento obrigatório neste catálogo:** `Money` (valor + moeda), `Cpf`, `Cnpj`, `AutorizacaoId`, `IdContrato`, `PartitionId`, `PurgeRange`, `PeriodoVigencia` (início + fim), `ChavePix`.

**[❌ Código Não Aderente]:**

```
public class PixBufferRingPartitionPurgeManager {
 public void executePurge(String particaoStr, String rangeStr) {
 // parsing fragil + regra de dominio solta em if
 int inicio = Integer.parseInt(rangeStr.split("-")[0]);
 int fim = Integer.parseInt(rangeStr.split("-")[1]);
 int part = Integer.parseInt(particaoStr);
 if (part >= inicio && part <= fim) {
 bufferRing.purge(part);
 }
 }
}
```

```
 }
}
}
```

#### [🔥 Violação e Explicação]:

1. **Primitive Obsession** (Refactoring Guru): `String` carregando semântica de range, parsing repetido em todo lugar, validação frágil (`split("-")` quebra com `"900-999-1000"`).
2. **Nomes não grepáveis**: `Handler/Manager` solto, parâmetros `p/rng` ilegíveis.
3. **Janela de contexto do LLM**: o agente precisa inferir o formato da string e a regra de range a cada leitura. Encapsular torna o tipo auto-documentado.

#### [✅ Exemplo de Refatoração]:

```
public record PartitionId(int value) {
 public PartitionId {
 if (value < 0) throw new IllegalArgumentException("partition deve ser >= 0");
 }
}

public record PurgeRange(int inicio, int fim) {
 public PurgeRange {
 if (inicio > fim) throw new IllegalArgumentException("range invalido");
 }
 public boolean contains(PartitionId p) {
 return p.value() >= inicio && p.value() <= fim;
 }
}

public class PixBufferRingPartitionPurgeManager {
 public void executePurge(PartitionId currentPartition, PurgeRange purgeRange) {
 if (purgeRange.contains(currentPartition)) {
 bufferRing.purge(currentPartition);
 }
 }
}
```

## Replace Magic Number with Symbolic Constant (regra de negócio)

Qualquer literal numérico ou `String` com **significado de domínio** é proibido no meio de validações, fórmulas ou `switch/if`. Deve virar constante nomeada, `enum` ou `value object`.

**Quando NÃO aplicar**: constantes matemáticas universais triviais (0, 1, 100 como percentual, índices de array) podem permanecer. A regra se aplica a literais com semântica de negócio desconhecida para quem não é SME do domínio.

#### [❌ Código Não Aderente]:

```
public void validarTransacaoPix(Conta conta, BigDecimal valor, int tipo) {
 if (valor.compareTo(new BigDecimal("50000")) > 0) { // 50000 = ?
 throw new BusinessException("valor excede limite");
 }
 if (tipo == 1) { // 1 = ?
 conta.sacar(valor);
 } else if (tipo == 2) { // 2 = ?
 conta.depositar(valor);
 }
}
```

#### [🔥 Violação e Explicação]:

1. **Magic Numbers** (Refactoring Guru + Object Calisthenics): o significado de 50000, 1 e 2 está oculto — o agente precisa adivinhar a regra de negócio.
2. **Acoplamento de mudança**: se o Banco Central alterar o limite regulatório, o agente tem que caçar 50000 no projeto inteiro (e pode errar um).
3. **Custo de janela de contexto**: cada literal exige uma volta ao domínio para entender.

#### [✅ Exemplo de Refatoração]:

```
// Limite regulado pelo Banco Central na resolucao BCB 123/2024, art. 7o §2o.
// Nao alterar sem alinhamento com compliance.
private static final BigDecimal LIMITE_MAXIMO_PIX_AUTOMATICO = new BigDecimal("50000");

public enum TipoTransacao { SAQUE, DEPOSITO }

public void validarTransacaoPix(Conta conta, Money valor, TipoTransacao tipo) {
 if (valor.isGreaterThan(LIMITE_MAXIMO_PIX_AUTOMATICO)) {
 throw new BusinessException(String.format(
```

```

 "Valor %s excede limite PIX Automatico de %s",
 valor, LIMITE_MAXIMO_PIX_AUTOMATICO));
 }
 switch (tipo) {
 case SAQUE -> conta.sacar(valor);
 case DEPOSITO -> conta.depositar(valor);
 }
}

```

**Por que enum em vez de int?** Porque o compilador garante exhaustive switch em `sealed types`, e o `rg "TipoTransacao.SAQUE"` cai direto onde o tipo é usado.

## Guard Clauses (early return, zero else)

O `else` é **proibido** neste catálogo. Use a forma positiva da guarda: `if (!condicao) return;` ou `throw`, deixando o corpo do método no mesmo nível de indentação. Cada nível de indentação extra multiplica o custo cognitivo do LLM para rastrear o estado da execução.

**Exceção:** o `else` é tolerado em `switch expressions` (Java 14+) e em pattern matching exaustivo de `sealed types`, que são esgotamento, não aninhamento.

### [❌ Código Não Aderente]:

```

public void processar(Pedido pedido) {
 if (pedido != null) {
 if (pedido.itens() != null) {
 if (!pedido.itens().isEmpty()) {
 if (pedido.valor().signum() > 0) {
 executar(pedido);
 } else {
 throw new BusinessException("valor invalido");
 }
 } else {
 throw new BusinessException("sem itens");
 }
 } else {
 throw new BusinessException("itens nulos");
 }
 }
}
}

```

### [🔥 Violação e Explicação]:

1. **Aninhamento profundo (4 níveis)** — pirâmide de `if/else` torna impossível seguir o fluxo principal sem perder o estado.
2. **Else explícito** — quando o `if` retorna/throw, o `else` é ruído.
3. **Método longo** — excede 20 linhas; viola regra de janela de contexto.

### [✅ Exemplo de Refatoração]:

```

public void processar(Pedido pedido) {
 if (pedido == null || pedido.itens() == null || pedido.itens().isEmpty()) {
 return;
 }
 if (pedido.valor().signum() <= 0) {
 throw new BusinessException("valor invalido");
 }
 executar(pedido);
}

```

## Clean Code for AI (tamanho e comentários)

Esta seção consolida as regras de "otimização para janela de contexto" que atravessam todas as outras — tamanho de método/arquivo, nomes grepáveis, comentários de proveniência e tipagem explícita.

**Tamanho de método:** 4-20 linhas. Acima disso, **Extract Method** até caber. Métodos longos escondem a lógica e fazem o agente perder o fio entre cláusulas.

**Tamanho de arquivo:** 300-500 linhas. Acima disso, dividir por responsabilidade (SRP). Arquivos grandes são truncados em diffs e forçam o agente a carregar contexto irrelevante.

**Comentários "por que", não "o que":** eliminar comentários redundantes (ex: `// incrementa i, // verifica se o saldo e maior que zero`) que gastam tokens reais. Preservar (e **exigir**) comentários de **proveniência** que explicam a decisão não-óbvia.

### [❌ Código Não Aderente]:

```

// ruído que come janela de contexto
// incrementa i
i++;
// verifica se o saldo e maior que zero
if (saldo.compareTo(BigDecimal.ZERO) > 0) { ... }

```

[🔥 **Violação e Explicação**]: `// incrementa i e // verifica se o saldo e maior que zero` são traduções literais do código — `git blame` + nome do método já respondem "o que". Gastam tokens e atrapalham a leitura do agente.

[✅ **Exemplo de Refatoração**]:

```
// comentario de proveniencia: explica decisao nao-obvia
// Limite regulado pelo Banco Central na resolucao BCB 123/2024, art. 7o §2o.
// Nao alterar sem alinhamento com compliance.
private static final BigDecimal LIMITE_MAXIMO_PIX_AUTOMATICO = new BigDecimal("50000");
```

**Regra prática**: se o `git blame` + nome do método já respondem "o que", o comentário é redundante. Se a regra veio de um ofício, um ADR ou um workaround de bug antigo, **esse** comentário precisa existir — é justamente o que o agente não consegue inferir.

**Tipagem explícita**: assinaturas devem ser fortemente tipadas. `Map/List/Set` sem tipo, `Object`, `String` para tudo, ou raw types obrigam o agente a inferir tipos a cada leitura.

[❌ **Código Não Aderente**]:

```
// raw type; agente precisa inferir
public Map buscar(String p) { ... }
public void executar(Object p) { ... }
```

[🔥 **Violação e Explicação**]: raw types e `Object` desativam o type checker; o agente precisa inferir tipos a cada leitura e não recebe proteção contra `ClassCastException` em runtime.

[✅ **Exemplo de Refatoração**]:

```
// tipos explicitos na assinatura
public Map<AutorizacaoId, Autorizacao> buscarPorFiltro(FiltroAutorizacao filtro) { ... }
public void executar(AutorizacaoParaExpirar autorizacao) { ... }
```

## Bloaters e Change Preventers (centralização de mudança)

Se uma única alteração de regra de negócio exige editar 20 arquivos diferentes, o código está mal distribuído. Tipos comuns a vigiar:

- **Primitive Obsession** (ver seção acima) — quando a regra de domínio está no `if` solto, mudar a regra exige varrer o projeto.
- **Shotgun Surgery** — uma feature nova precisa tocar 5 classes? Falta um *aggregate root* ou *use case* que concentre a operação. Ver arquitetura-limpa-java.
- **Divergent Change** — uma classe muda por motivos não-relacionados? Extrair por responsabilidade (SRP).
- **Configuração dispersa** — `@Value("${limite.pix}")` espalhado por 10 arquivos? Mover para um único `@ConfigurationProperties` injetado por construtor.

## Refactorings do Fowler — guia rápido

Catálogo dos refactorings mais comuns em Java moderno, com exemplo ❌/🔥/✅ unificado com a skill `revisao-de-codigo-java`. Cada refactoring resolve um **cheiro** (code smell) específico — não aplique por aplicar.

**Mapeamento smell → refactoring**: Tell-Don't-Ask, Primitive Obsession, Magic Numbers e Guard Clauses têm seções dedicadas acima. Esta parte cobre os refactorings mecânicos (Remove Parameter, Extract Method, Replace Conditional with Polymorphism, etc.).

**Formato único**: toda esta skill usa exclusivamente o padrão ❌ Código Não Aderente / 🔥 Violação e Explicação / ✅ Exemplo de Refatoração — mesmo formato da `revisao-de-codigo-java`. Quando o `java-revisor` reporta um achado, basta copiar o bloco ❌/🔥/✅ desta skill para dentro do relatório (preenchendo com o caso real).

### Remove Parameter

**Quando**: um parâmetro nunca é usado, ou seu valor pode ser obtido de outro lugar (campo da classe, constante, chamada de método).

[❌ **Código Não Aderente**]:

```
// "isCloud" e recebido mas nunca influencia o resultado
public Backend selecionarBackend(long tableId, ConnectContext context, boolean isCloud) {
 return sistemaInfo.getBackend(selecionarBackendInterno(tableId, context.getCluster()));
}
```

[🔥 **Violação e Explicação**]: parâmetro morto infla a assinatura, confunde o caller sobre qual valor passar, e é candidato permanente a "ser usado no futuro" — abstração especulativa (YAGNI).

[✅ **Exemplo de Refatoração**]:

```
public Backend selecionarBackend(long tableId, ConnectContext context) {
 return sistemaInfo.getBackend(selecionarBackendInterno(tableId, context.getCluster()));
}
```

Veja a skill dedicada `refactoring-remove-parameter` para a versão focada e passo-a-passo desse refactoring.

## Extract Method

**Quando:** um trecho de código tem um propósito claro e pode ser nomeado, ou você quer reusá-lo. **Regra prática:** método com mais de 20 linhas, ou que misture "preparar/validar" com "executar", sempre tem um `Extract Method` a oferecer.

### [❌ Código Não Aderente]:

```
public void processar(Pedido pedido) {
 if (pedido.getValor() == null || pedido.getValor().signum() <= 0) {
 throw new BusinessException("Valor invalido");
 }
 if (pedido.getItems() == null || pedido.getItems().isEmpty()) {
 throw new BusinessException("Pedido sem itens");
 }
 // ... logica principal começa aqui
}
```

[🔥 **Violação e Explicação:** método com 2 responsabilidades (validar + executar) e > 20 linhas; validação inline polui o fluxo principal e impede teste isolado da regra.

### [✅ Exemplo de Refatoração]:

```
public void processar(Pedido pedido) {
 validar(pedido);
 // ... logica principal começa aqui
}

private void validar(Pedido pedido) {
 if (pedido.getValor() == null || pedido.getValor().signum() <= 0) {
 throw new BusinessException("Valor invalido");
 }
 if (pedido.getItems() == null || pedido.getItems().isEmpty()) {
 throw new BusinessException("Pedido sem itens");
 }
}
```

## Replace Magic Number with Symbolic Constant

**Caso geral (constantes técnicas):** retry, timeouts, tamanhos de página. Já coberto pela seção "Replace Magic Number (regra de negócio)" acima quando o número tem semântica de domínio.

### [❌ Código Não Aderente]:

```
// constante tecnica repetida em mais de um lugar
if (tentativas > 3) { ... }
Thread.sleep(1000L);
```

[🔥 **Violação e Explicação:** 3 e 1000L com significado técnico (max tentativas, intervalo) mas literais soltos; ao ajustar, o agente tem que caçar os números pelo projeto.

### [✅ Exemplo de Refatoração]:

```
private static final int MAX_TENTATIVAS = 3;
private static final long INTERVALO_RETRY_MS = 1_000L;

if (tentativas > MAX_TENTATIVAS) { ... }
Thread.sleep(INTERVALO_RETRY_MS);
```

## Replace Conditional with Polymorphism

**Quando:** um `switch/if` chain decide por **tipo** e cada ramo tem lógica distinta.

### [❌ Código Não Aderente]:

```
// instanceof chain decide por tipo, com throw generico para tipo desconhecido
public BigDecimal calcularTaxa(Pagamento pagamento) {
 if (pagamento instanceof Pix) return BigDecimal.ZERO;
 if (pagamento instanceof Cartao) return BigDecimal.valueOf(0.03);
 throw new IllegalStateException("Tipo desconhecido");
}
```

[🔥 **Violação e Explicação:** `instanceof` chain é aberta a extensão (cada novo tipo exige editar o método) e o `throw` final só é detectado em runtime; o compilador não ajuda a lembrar todos os tipos.

### [✅ Exemplo de Refatoração]:

```
// sealed type + switch exaustivo (ver java-moderno)
public BigDecimal calcularTaxa(Pagamento pagamento) {
 return switch (pagamento) {
```



```
 case Pix p -> BigDecimal.ZERO;
 case Cartao c -> BigDecimal.valueOf(0.03);
 // compilador exige todos os casos se Pagamento for sealed
 };
}
```

Introduce Parameter Object

**Quando:** um grupo de parâmetros viaja junto em vários métodos (é o oposto de "Primitive Obsession" — aqui o grupo é heterogêneo, mas sempre os mesmos campos).

[❌ Código Não Aderente]:

```
// grupo de 4 parametros viaja junto em varios metodos
public void buscar(LocalDate inicio, LocalDate fim, String status, int pagina) { ... }
public void exportar(LocalDate inicio, LocalDate fim, String status) { ... }
```

[🔥 Violação e Explicação]: os mesmos 4 parâmetros se repetem; adicionar/remover um campo exige editar a assinatura de cada método e cada caller; propensão a erros de ordem (trocar `inicio` por `fim`).

[✅ Exemplo de Refatoração]:

```
public record FiltroPedido(LocalDate inicio, LocalDate fim, String status) {}

public void buscar(FiltroPedido filtro, int pagina) { ... }
public void exportar(FiltroPedido filtro) { ... }
```

Replace Loop with Pipeline

**Quando:** um loop acumula resultado em uma coleção com transformações triviais.

[❌ Código Não Aderente]:

```
// loop mutando lista externa
List<String> nomes = new ArrayList<>();
for (Produto p : produtos) {
 if (p.isAtivo()) {
 nomes.add(p.getNome().toUpperCase());
 }
}
```

[🔥 Violação e Explicação]: loop imperativo com mutação; impede paralelização futura e é mais verboso que um pipeline curto para transformações triviais.

[✅ Exemplo de Refatoração]:

```
List<String> nomes = produtos.stream()
 .filter(Produto::isAtivo)
 .map(p -> p.getNome().toUpperCase())
 .toList();
```

Ver `revisao-de-codigo-java` (item 4 — Streams) e `java-moderno` (seção Stream) para quando loop é preferível a pipeline (clareza > "tudo em stream").

Quem aplica o quê

Situação	Quem	Skill
Aplicar refactoring em uma classe/método	sessão principal	esta skill
Revisar diff/PR com checklist de severidade	agent java-revisor	revisao-de-codigo-java
Remoção de parâmetro focada (passo-a-passo)	sessão principal	refactoring-remove-parameter
Limpar imports não usados	sessão principal	remover-imports-nao-usados
Centralizar configuração dispersa (Shotgun Surgery)	session/engenheiro-devops	java-architecture
Decidir onde mora um value object novo	session/java-construtor	arquitetura-limpa-java